Prototipo de sistema de traducción de Lengua de Señas Argentina con procesamiento en tiempo cuasi real

Maite Nigro Francisco Veron

Director: Esp. Ing. Gonzalo Avalos Ribas

Co-director: Ing. Bruno Constanzo

Universidad Nacional de Mar del Plata
Facultad de Ingeniería
Departamento de Ingeniería en Informática

Marzo 2026

Agradecimientos

De Maite: Para la Flaqui.
De Francisco: Para el Diego y Baco.
A Miriam Rolls, por su orientación sobre la LSA y la validación del vocabulario del conjunto de datos.

Índice general

1. Introducción **1**
- 1.1. Contexto . . . 1
- 1.2. Motivación . . . 1
- 1.3. Problemática . . . 1
- 1.4. Alcance y limitaciones . . . 2
  - 1.4.1. Delimitación del vocabulario y asesoramiento lingüístico . . . 2
  - 1.4.2. Límites operativos y comportamiento ante entradas no contempladas 3
  - 1.4.3. Infraestructura de despliegue . . . 3

2. Objetivos **4**
- 2.1. Objetivo general . . . 4
- 2.2. Objetivos específicos . . . 4

3. Marco teórico, estado del arte y metodología **6**
- 3.1. Marco teórico . . . 6
  - 3.1.1. Componentes articulatorios y estructura fonológica de la LSA . . . 6
  - 3.1.2. Estructura lingüística de la LSA y traducción *Topic-Comment* . . . 6
  - 3.1.3. Sistemas de tiempo real y tiempo pseudo-real . . . 7
  - 3.1.4. Visión computacional y reconocimiento basado en esqueletos . . . 7
  - 3.1.5. Modelado de secuencias temporales y atención . . . 8
  - 3.1.6. Modelos de lenguaje pequeños y adaptación *LoRA* . . . 8
  - 3.1.7. Estándares de captura web y arquitecturas de extensión web . . . 9
- 3.2. Estado del arte . . . 10
  - 3.2.1. Desarrollos en Argentina y la región . . . 10
  - 3.2.2. Sistemas internacionales de reconocimiento y síntesis . . . 10
- 3.3. Metodología de desarrollo . . . 10
  - 3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial . . 11
- 3.4. Herramientas y tecnologías empleadas . . . 12

4. Análisis de requerimientos **13**
- 4.1. Requerimientos funcionales . . . 13
- 4.2. Requerimientos no funcionales . . . 14
- 4.3. Restricciones . . . 14
  - 4.3.1. Especificación del entorno de ejecución cliente . . . 14
- 4.4. Casos de uso . . . 15
  - 4.4.1. Casos de uso esenciales en formato breve . . . 15
  - 4.4.2. Casos de uso expandidos . . . 16

5. Diseño del Sistema **20**
5.1. Arquitectura del sistema . . . 20
5.1.1. Justificación de la partición y modelo de privacidad . . . 20
5.1.2. Flujo integral de datos entre componentes. . . . 20
5.2. Diseño de contratos de API REST . . . 20
5.2.1. API local del motor ILSA . . . 21
5.2.2. API del servicio semántico remoto . . . 25
5.2.3. API del servicio de archivos del avatar . . . 27
5.3. Diseño de la representación gestual inversa . . . 28
5.3.1. Clip de referencia . . . 29
5.3.2. Preparación y empalme . . . 29
5.3.3. Presentación en la videollamada . . . 31
5.3.4. Contingencia léxica . . . 32
5.4. Diseño de la extensión de videollamada . . . 33
5.4.1. Frontera . . . 33
5.4.2. Dos vistas de la misma llamada . . . 33
5.4.3. Aislamiento . . . 34
5.4.4. Fallo de un servicio . . . 35
5.5. Patrones de diseño de software aplicados . . . 35
5.6. Diseño conceptual de la arquitectura escalable de producción en la nube . . . 36
5.6.1. Ingreso . . . 36
5.6.2. Traductor . . . 36
5.6.3. Archivos de seña . . . 36
5.6.4. Monitoreo . . . 36
5.6.5. Qué queda fuera de las pruebas . . . 37

6. Implementación y Desarrollo **38**
6.1. Implementación del subsistema de reconocimiento gestual . . . 38
6.1.1. Preprocesamiento de datos . . . 38
6.1.2. Preprocesamiento y arquitectura neuronal TinySkeletonClassifier . . . 38
6.2. Implementación del subsistema de traducción semántica bidireccional . . . 39
6.2.1. Selección y especificación de la suite de modelos de lenguaje . . . 39
6.2.2. Régimen común del entrenamiento . . . 39
6.2.3. Canal directo (LSA → Español) . . . 39
6.2.4. Canal inverso (Español → LSA) . . . 40
6.2.5. Cuantización GGUF Q4_K_M y servicio de inferencia . . . 41
6.3. Integración en Google Meet y extensión de navegador . . . 41
6.3.1. Despliegue en Chrome Manifest V3: aislamiento offscreen y sandbox 42
6.3.2. Intercepción de la captura de cámara y lienzo virtual . . . 42
6.3.3. Motor local ILSA (FastAPI) . . . 43
6.4. Implementación del avatar . . . 44
6.4.1. Tecnologías empleadas . . . 44
6.4.2. Vía descartada: retarget hacia un avatar VRM . . . 44
6.4.3. Estructura del pedido . . . 45
6.4.4. Decisiones técnicas de la reproducción . . . 45
6.5. Instancia de validación del prototipo mediante túnel seguro . . . 46

7. Experimentación, Resultados y Discusión **47**
7.1. Corpus y metodología de adquisición de Datos de LSA 47
7.1.1. Definición y validación del vocabulario de 100 clases 47
7.1.2. Protocolo de grabación y curación del dataset propio 47
7.2. Evaluación empírica del clasificador gestual 48
7.2.1. Métricas offline vs. rendimiento en cámara real 48
7.2.2. Impacto de la resolución temporal: 16 vs. 32 fotogramas 49
7.2.3. Búsqueda automatizada de hiperparámetros con Optuna y Análisis de sobreajuste 50
7.2.4. Diagnóstico de confusiones mediante feature engineering y separabilidad 51
7.2.5. Impacto cuantitativo de la corrección de aspect ratio 52
7.2.6. Desempeño final del clasificador baseline en vivo 52
7.3. Evaluación de los módulos de traducción semántica 55
7.3.1. Benchmark comparativo de modelos SLM (Familias Qwen2.5, Llama 3.2 y SmolLM2) 55
7.3.2. Evaluación cuantitativa: BLEU, ROUGE-L, METEOR y exactitud 55
7.3.3. Análisis de simetría en el aprendizaje: traducción directa vs. estructuración inversa 55
7.3.4. Compromiso entre calidad lingüística y latencia de inferencia 55
7.4. Evaluación del sistema integrado y rendimiento en tiempo real 55
7.4.1. Validación funcional de extremo a extremo en videollamadas reales 55
7.4.2. Análisis de latencias totales por etapa del pipeline 55
7.4.3. Consumo de recursos de hardware: máquina de desarrollo vs. máquina objetivo 55
7.4.4. Discusión técnica y lecciones de ingeniería aprendidas 55

8. Conclusiones y trabajo futuro **56**
8.1. Conclusiones 56
8.1.1. Cumplimiento de objetivos del proyecto 56
8.1.2. Aportes técnicos y de accesibilidad 56
8.1.3. Impacto social en el marco de la Ley 27.710 56
8.2. Trabajo futuro 56
8.2.1. Implementación física del clúster semántico escalable 56
8.2.2. Expansión del corpus multiseñante y mitigación de variabilidad . . 56
8.2.3. Incorporación de rasgos no manuales (expresiones faciales) 56
8.2.4. Generalización a nuevas plataformas de comunicación virtual 56

A. Vocabulario del clasificador **59**

C. Arquitectura escalable de producción en la nube **62**

Resumen

En la actualidad, la interacción virtual entre personas sordas y oyentes se limita casi exclusivamente a la comunicación escrita o a la transcripción unilateral de voz a texto, lo que genera una marcada asimetría en la comunicación expresiva durante videollamadas. Este Trabajo Final de Grado desarrolla un prototipo de traducción bidireccional entre la Lengua de Señas Argentina (LSA) y el español oral estructurado bajo una arquitectura híbrida de computación en el borde y servicios en la nube con procesamiento en tiempo pseudo-real.

El sistema articula dos canales comunicacionales complementarios. En el sentido señante-oyente, la computadora del usuario captura localmente el flujo de video desde la cámara web, extrae coordenadas articulares tridimensionales mediante visión computacional y clasifica un vocabulario de 100 señas mediante una red neuronal ligera (Conv1D + Tiny Transformer con *attention pooling*). Con el fin de evitar la saturación de memoria y cómputo que provocaría ejecutar modelos de lenguaje generativos en dispositivos convencionales, las glosas clasificadas son enviadas hacia un backend semántico en la nube que transforma la secuencia en oraciones fluidas en español rioplatense, las cuales retornan al cliente para su proyección en subtítulos o síntesis de voz. En sentido inverso (oyente-señante), el sistema procesa la voz y/o texto del interlocutor oyente mediante un módulo semántico inverso en la nube que estructura las glosas, enviándolas a un avatar en el cliente que ejecuta las animaciones correspondientes o recurre al deletreo dactilológico para términos fuera del léxico.

El módulo clasificador gestual local alcanza un 98,28 % de exactitud en validación fuera de línea y un 77,3 % de acierto en cámara real. Por su parte, los modelos de lenguaje evaluados (Qwen2.5, Llama 3.2 y SmolLM adaptados mediante LoRA y cuantizados a GGUF Q4_K_M) superan el 94 % de exactitud y alcanzan latencias de inferencia de entre 200 y 450 ms. Como mecanismo de integración para videollamadas, el sistema se acopla a Google Meet mediante una extensión de navegador conectada al motor local ILSA, y asegura la privacidad del usuario al no transmitir video crudo fuera del dispositivo cliente. Finalmente, se deja documentada la propuesta de diseño de una arquitectura en la nube escalable para soportar la demanda concurrente en entornos de producción.

Palabras clave:

Lengua de Señas Argentina, reconocimiento de gestos, arquitectura híbrida cliente-nube, traducción semántica bidireccional, avatar 3D, accesibilidad.

Capítulo 1

Introducción

1.1. Contexto

Tradicionalmente, la sociedad abordó a la comunidad sorda desde una perspectiva clínica centrada en la rehabilitación y la corrección auditiva. Ese enfoque omite que la comunidad sorda se identifica, ante todo, como una minoría lingüística con cultura y gramática propias, independientes del español oral. En la Argentina, la Lengua de Señas Argentina (LSA) cuenta con reconocimiento legal formal a través de la Ley 27.710 (República Argentina, 2023), la cual garantiza su preservación y el derecho de las personas sordas a comunicarse en su lengua natural en todos los ámbitos sociales.

En el contexto digital contemporáneo, las plataformas de videoconferencia se han convertido en herramientas centrales para el ámbito laboral, educativo y de la vida cotidiana. Sin embargo, los *software* de uso masivo presentan escasa integración de herramientas automatizadas que admitan la LSA como un flujo comunicacional bidireccional en tiempo pseudo-real. Los recursos tecnológicos existentes suelen restringirse a diccionarios estáticos o repositorios de video, carentes de capacidad de interpretación contextual e interactiva durante una llamada.

1.2. Motivación

Este proyecto nace de la aspiración de reducir las barreras comunicacionales de las personas señantes en interacciones cotidianas y de asistencia mediante herramientas informáticas. Resulta indispensable avanzar hacia esquemas de integración lingüística en los que la tecnología actúe como un puente transparente en ambos sentidos de la conversación: expresivo (señante $\rightarrow$ oyente) y receptivo (oyente $\rightarrow$ señante).

1.3. Problemática

La problemática central radica en la asimetría comunicacional presente en las plataformas de videoconferencia. Si bien las herramientas de transcripción automática de voz a texto han alcanzado un alto grado de adopción, resultan insuficientes como solución de accesibilidad integral: para diversas personas sordas, el texto escrito opera como una segunda lengua con reglas gramaticales ajenas, e incluso genera barreras de acceso para quienes no se encuentran alfabetizados en la lengua oral.

En el extremo expresivo, la comunicación mediante LSA carece de soporte nativo en el software de videollamadas comercial. La falta de módulos capaces de capturar e interpretar la LSA hacia voz o texto audible fuerza la dependencia de interpretación humana presencial o restringe la interacción al chat escrito.

A esta brecha se añade la especificidad lingüística local: los modelos de visión y lenguaje desarrollados internacionalmente se encuentran enfocados mayoritariamente en la Lengua de Señas Americana (ASL). Las técnicas y arquitecturas neuronales resultan reutilizables, pero los modelos preentrenados y conjuntos de datos de ASL no se extrapolan directamente a la LSA debido a diferencias léxicas, dactilológicas y sintácticas sustanciales, por lo que es necesaria la construcción de un corpus propio y de modelos adaptados al contexto argentino.

1.4. Alcance y limitaciones

El sistema desarrollado comprende un prototipo funcional de traducción bidireccional en tiempo pseudo-real concebido como una extensión para navegadores basados en la tecnología Chromium (como Google Chrome, Microsoft Edge, Brave, Opera o Vivaldi), articulada con un motor de ejecución en segundo plano denominado ILSA (*Intérprete de LSA*).

Para abordar la interpretación de la LSA se introduce el concepto de **glosa**: representación escrita en letras mayúsculas asignada a una seña para identificarla, y analizar su gramática sin constituir una traducción literal término a término.

El sistema opera a través de dos canales comunicacionales integrados:

- **Canal directo (Señante → Oyente):** En el cliente, el sistema toma señas aisladas de la cámara, extrae coordenadas articulares y clasifica cada glosa. La pausa que cierra el enunciado vale 4 segundos por defecto; la extensión también admite 2 o 3. Una sola glosa léxica, un deletreo de letras o una seguidilla de dígitos se resuelven en ILSA y no consultan la nube. El resto viaja como texto de glosas al servicio semántico, que devuelve una oración en español rioplatense. La extensión la muestra como subtítulo dentro del video, la lee en voz alta en la misma computadora, o hace ambas cosas. Esa voz no entra como pista de audio de la llamada. Si el servicio no responde, ILSA muestra las glosas en texto, sin rearmar la oración.
- **Canal inverso (Oyente → Señante):** La voz del oyente se transcribe en su computadora, o el sistema toma el texto del chat. La oración va a la nube, que la reordena en glosas de LSA. El cliente dibuja el clip de pose de cada glosa sobre un esqueleto. Si la glosa no tiene archivo, ILSA pide cada letra por separado y la dibuja con el alfabeto manual.

1.4.1. Delimitación del vocabulario y asesoramiento lingüístico

El alcance funcional del clasificador gestual está delimitado a un vocabulario objetivo de 100 señas de LSA (cuyo detalle y categorización se presentan en el Apéndice A). Para la selección y estructuración de este corpus se contó con la colaboración y asesoramiento técnico de Miriam Rolls, profesora e intérprete de LSA, quien participó en la delimitación del recorte léxico. El vocabulario se orientó hacia situaciones de interacción cotidiana y contextos de primera respuesta o emergencia (verbos de auxilio, identificación, parentesco y datos temporales). Esas señas se distinguen sin la gesticulación facial. La validación

garantizó la fidelidad lingüística de las señas seleccionadas para la construcción del corpus, sin que esto constituya una evaluación formal de usabilidad ni un ensayo clínico con usuarios finales en entornos reales de emergencia.

1.4.2. Límites operativos y comportamiento ante entradas no contempladas

El prototipo presenta las siguientes delimitaciones operativas:

- **Delimitación de señas:** El comienzo y fin de una seña se determina mediante la deteccion o ausencia de una mano en el frame.
- **Delimitación temporal de enunciados:** El cierre de una frase de señas ocurre tras una pausa sin actividad gestual. El valor inicial es 4 segundos. La extensión también permite 2 o 3 segundos.
- **Rasgos no manuales:** El clasificador excluye la malla facial y no trata la boca, las cejas ni la cabeza como rasgo gramatical.
- **Entradas con baja confianza o fuera del vocabulario:** En el canal directo, ILSA descarta la glosa si la confianza queda bajo 0,75. Un segundo filtro rechaza la repetición inmediata de una glosa léxica, admite la repetición en los dígitos y admite hasta dos letras iguales seguidas. En el canal inverso, una palabra sin archivo de pose se deletrea letra por letra. Ese deletreo muestra los caracteres y no asegura, por sí solo, que el interlocutor sordo conozca el vocablo en español.
- **Resiliencia ante fallas de red:** En caso de pérdida de conectividad con el servicio semántico remoto, el motor local ILSA conmuta a un modo de contingencia que proyecta las glosas clasificadas en texto directo sin reestructuración sintáctica.

1.4.3. Infraestructura de despliegue

En la red, el alcance incluye el diseño de una arquitectura elástica, con contenedores y balanceo de carga, para varias videollamadas a la vez. Las pruebas no despliegan ese esquema. La evaluación usa una instancia de Cloud Run por servicio, en CPU.

Capítulo 2

Objetivos

2.1. Objetivo general

Desarrollar un prototipo de sistema de traducción bidireccional entre la LSA y el español rioplatense con procesamiento en tiempo pseudo-real, diseñado para operar sobre plataformas de videoconferencia mediante una arquitectura híbrida cliente-nube, con el fin de mitigar la asimetría en la comunicación expresiva y receptiva, lo que ayudaría a fomentar la autonomía de las personas sordas en la interacción con interlocutores oyentes no señantes.

2.2. Objetivos específicos

- Implementar un módulo local de visión computacional y clasificación gestual basado en la extracción de puntos de referencia corporales para el reconocimiento de un vocabulario de 100 señas de LSA.
- Diseñar e integrar el servicio semántico del canal directo (LSA → Español) para transformar secuencias de glosas aisladas en oraciones fluidas y gramaticalmente correctas en español rioplatense.
- Diseñar e integrar el servicio semántico del canal inverso (Español → LSA) para reestructurar oraciones del español oral a secuencias de glosas alineadas con las reglas morfosintácticas y la estructura canónica *Topic-Comment* de la LSA.
- Implementar un módulo de visualización gestual inversa mediante un avatar capaz de animar las señas resultantes y ejecutar deletreo dactilológico para términos ausentes en el vocabulario.
- Diseñar una arquitectura escalable en la nube para el aprovisionamiento concurrente de los servicios semánticos, lo que desacopla el procesamiento del hardware cliente.
- Resguardar la privacidad de los datos biométricos de los usuarios al ejecutar la extracción de características y la clasificación de forma local en el dispositivo cliente, y, por consiguiente, limitar la transmisión hacia la nube a cargas puramente textuales.
- Desarrollar la arquitectura de software de una extensión para navegadores basados en la tecnología Chromium (tales como Google Chrome, Microsoft Edge, Brave, Opera o Vivaldi) articulada con un motor de ejecución en segundo plano (ILSA).

- Evaluar el desempeño integral del prototipo en términos de exactitud de clasificación con personas no vistas en el entrenamiento, fidelidad semántica bidireccional (métricas BLEU y ROUGE-L), latencia de respuesta y consumo de recursos de hardware, conforme a las métricas y criterios de aceptación definidos en el marco metodológico.

Capítulo 3

Marco teórico, estado del arte y metodología

3.1. Marco teórico

3.1.1. Componentes articulatorios y estructura fonológica de la LSA

La articulación de la LSA se sostiene sobre la interacción simultánea de dos grandes conjuntos de parámetros lingüísticos:

- **Rasgos manuales:** Comprenden la configuración o forma de la mano, la ubicación o punto de articulación en el espacio corporal, la orientación de la palma y la trayectoria cinemática (Massone & Martínez, 2012).
- **Rasgos no manuales:** Incluyen la gesticulación facial (movimiento de cejas, boca y mirada), la inclinación del torso y la postura del cuello (Massone & Martínez, 2012). Estos elementos cumplen un rol determinante para matizar la intensidad de una acción, marcar interrogaciones o delimitar la polaridad en enunciados complejos.

Desde la perspectiva de la fonología gestual, la ejecución temporal de toda seña o glosa se estructura en tres fases secuenciales delimitadas en la literatura lingüística (**liddell1989phonological**):

- **Preparación (*Movement*):** Transición cinemática desde la postura de reposo o neutra hasta el punto de articulación inicial.
- **Núcleo (*Hold/Stroke*):** Fase central en la cual la configuración manual, la localización espacial y el movimiento otorgan el valor fonológico distintivo a la glosa.
- **Retracción (*Retraction*):** Retorno cinemático desde el término de la seña hacia la postura neutra de reposo.

3.1.2. Estructura lingüística de la LSA y traducción *Topic-Comment*

A nivel morfosintáctico, la LSA presenta diferencias sustanciales con el español rioplatense:

- **Estructura de la oración:** Mientras que el español sigue preferentemente un orden Sujeto-Verbo-Objeto (SVO), la LSA organiza sus proposiciones bajo el principio de *Topic-Comment* (Massone & Martínez, 2012).
- **Ausencia de elementos funcionales:** La LSA carece de artículos determinados e indeterminados («el», «la», «un»), preposiciones gramaticales abstractas («de», «a», «con») y verbos copulativos («ser» o «estar») (Massone, 1993; Massone & Martínez, 2012).
- **Marcadores de cierre:** Las partículas interrogativas (`QUÉ`, `DÓNDE`, `CUÁNDO`) y los operadores de negación (`NO`) se posponen obligatoriamente al final de la oración.

Bajo el esquema de *Topic-Comment*, el enunciado sitúa en primer lugar el marco temporal y espacial de referencia (el tópico o contexto), seguido por el sujeto, el objeto y finalmente la acción o estado junto con sus modificadores (el comentario):

$$\begin{aligned}\text{Oración LSA} \rightarrow \text{Tiempo} \rightarrow \text{Lugar} \rightarrow \text{Sujeto} \\ \rightarrow \text{Objeto/Sustantivo} \rightarrow \text{Verbo} \rightarrow \text{Negación/Pregunta}\end{aligned} \tag{3.1}$$

Debido a estas divergencias morfosintácticas, la equivalencia entre ambas lenguas no admite un mapeo léxico directo o traducción término a término.

3.1.3. Sistemas de tiempo real y tiempo pseudo-real

El concepto de **tiempo pseudo-real** designa a aquellos sistemas de procesamiento interactivo en los cuales la respuesta computacional no ocurre de forma estrictamente instantánea ni en la escala de los microsegundos, pero cuyos retardos temporales permanecen dentro de un umbral operativo tolerable para la comunicación humana. A diferencia de los sistemas de tiempo real estricto, el tiempo pseudo-real admite una latencia acotada producto de la segmentación temporal por pausas de inactividad, el empaquetado de datos y el tiempo de inferencia de los modelos semánticos en la nube, lo cual permite sostener la fluidez del diálogo en la videollamada.

3.1.4. Visión computacional y reconocimiento basado en esqueletos

El reconocimiento de señas a partir de flujo de video crudo opera, en cada fotograma, sobre millones de valores cromáticos. Un cuadro de 1920×1080 píxeles en espacio RGB contiene:

$$1920 \times 1080 \times 3 = 6\,220\,800 \tag{3.2}$$

escalares. Esta representación resulta altamente sensible a las variaciones de iluminación, al fondo y a la vestimenta del usuario. El reconocimiento basado en esqueletos reemplaza esta representación densa mediante la clasificación de un conjunto reducido de coordenadas articulares, denominadas *landmarks*.

Para la estimación postural se utiliza *MediaPipe Holistic*, una arquitectura de percepción distribuida para dispositivos de borde (Google, 2020; Lugaresi et al., 2019). Este marco no constituye un detector único, sino una serie de modelos preentrenados encadenados, que se ejecuta de forma local en el cliente sin transmitir la imagen cruda. El primer modelo, *BlazePose*, localiza al sujeto y estima 33 puntos de pose corporal (Bazarevsky et al., 2020).

Dichos puntos abarcan la estructura facial gruesa, el torso y las extremidades. A partir de las coordenadas de las muñecas, el pipeline delimita una región de interés (*ROI*) para cada mano, donde el modelo específico de manos (*hand*) infiere 21 puntos tridimensionales por mano (la muñeca y cuatro articulaciones por dedo) (Zhang et al., 2020).

Asimismo, los datos cinemáticos de referencia gestual se archivan en un formato binario especializado con extensión `.pose` (**moryossef2021poseformat**). La especificación del formato y su esquema de serialización provienen de la biblioteca *pose-format*, desarrollada para el almacenamiento y manipulación de datos posturales en lenguas de señas, la cual organiza los puntos en un encabezado de metadatos globales y un tensor multidimensional de coordenadas espacializadas.

3.1.5. Modelado de secuencias temporales y atención

La clasificación gestual requiere procesar secuencias temporales de coordenadas articulares para asignar una clase o glosa. La aplicación de modelos recurrentes convencionales, tales como las redes LSTM, presenta la limitación de comprimir la información histórica en un estado oculto secuencial, lo cual puede diluir los instantes iniciales del gesto al alcanzar la pose final.

El mecanismo de atención elimina esta restricción al permitir que cualquier instante temporal consulte de manera directa la totalidad de la secuencia en una única operación matricial (Vaswani et al., 2017). Para cada entrada, el modelo proyecta tres vectores: consulta (Q), clave (K) y valor (V). La respuesta atencional se calcula mediante la relación:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{3.3}$$

donde d_k representa la dimensión de la cabeza de atención. Este cálculo habilita al clasificador a ponderar las entradas de mayor relevancia informativa (como la pose sostenida durante el núcleo) por encima de las transiciónes.

3.1.6. Modelos de lenguaje pequeños y adaptación *LoRA*

Para la traducción semántica bidireccional se emplean modelos de lenguaje pequeños (SLM). La adaptación de estos modelos se ejecuta mediante la técnica de Adaptación de Bajo Rango (*LoRA*) (Hu et al., 2022), la cual congela la matriz de pesos preentrenados $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ e introduce dos matrices entrenables de bajo rango $\mathbf{B} \in \mathbb{R}^{d\times r}$ y $\mathbf{A} \in \mathbb{R}^{r\times k}$, con rango $r \ll \text{mín}(d, k)$. La actualización lineal adopta la forma:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{BA}) \tag{3.4}$$

donde α actúa como un factor de escala constante. Esta descomposición reduce en más de un 99 % la cantidad de parámetros optimizables sin comprometer la capacidad expresiva del modelo base (Hu et al., 2022).

Tras el proceso de ajuste, los modelos se convierten al formato GGUF mediante el esquema de cuantización `Q4_K_M` (Gerganov, 2023). En este esquema, los tensores se dividen en superbloques de 256 pesos y bloques menores con escala s y mínimo m propios. La reconstrucción del peso responde a la ecuación:

$$\hat{w} = m + s\, q \tag{3.5}$$

donde q representa el entero cuantizado de 4 bits. Esta técnica disminuye el consumo de memoria y habilita la inferencia de los modelos de lenguaje sobre arquitectura de CPU.

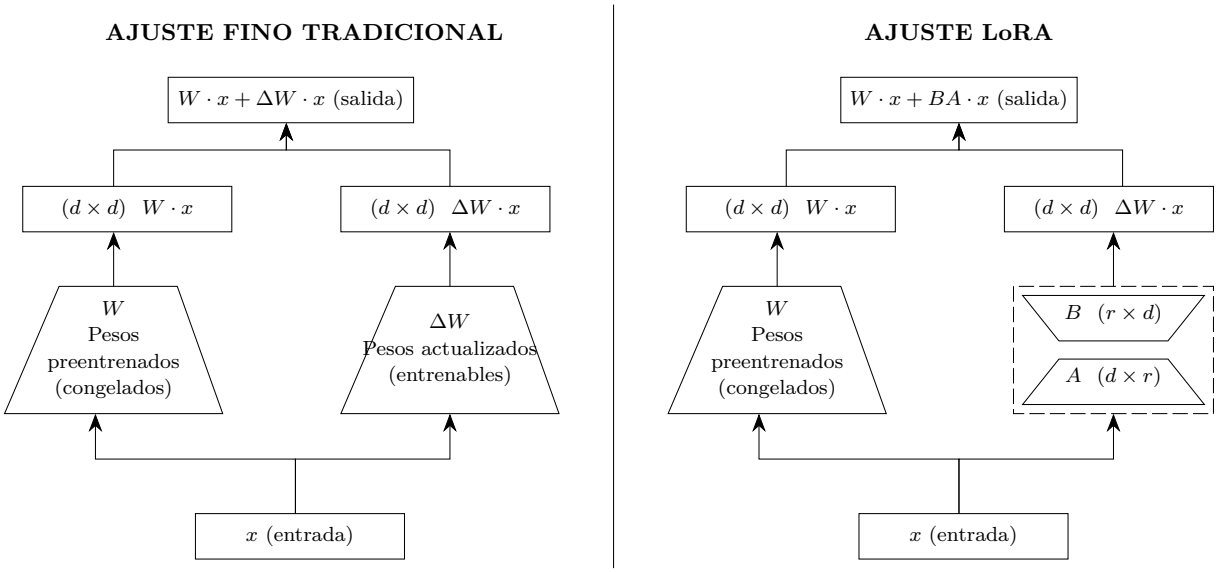


Figura 3.1: Comparación de arquitecturas: en el ajuste fino tradicional (izquierda) los pesos preentrenados W permanecen congelados y se actualiza de forma directa la matriz ΔW de dimensión $d \times d$. En el ajuste LoRA (derecha), la actualización de pesos ΔW se descompone en dos matrices entrenables de bajo rango A $(d \times r)$ y B $(r \times d)$, con $r \ll d$.

3.1.7. Estándares de captura web y arquitecturas de extensión web

La captura y procesamiento de video en el navegador web se fundamentan en la especificación W3C Media Capture (W3C, 2024). La interfaz nativa `getUserMedia()` requiere acceso exclusivo al dispositivo de cámara física. Para resolver esta limitación sin interrumpir el flujo de la videoconferencia, el método `canvas.captureStream()` de la especificación *Media Capture from DOM Elements* (**w3c2023mediacapturefromelement**) sintetiza un objeto `MediaStream` dinámico a partir de un lienzo virtual (`<canvas>`).

En el ámbito de las extensiones web bajo el estándar Manifest V3 (Google Chrome Developers, 2024), la Política de Seguridad de Contenido (CSP) prohíbe la ejecución de código dinámico en scripts de fondo. Para ejecutar los clasificadores y estimadores de visión computacional se implementa una arquitectura desacoplada en dos entornos aislados:

- **Documento en segundo plano (*offscreen*):** Entorno habilitado por la API `chrome.offscreen` (**chrome2024offscreen**) que provee contexto de DOM completo para coordinar la mensajería y gestionar lienzos desacoplados.
- **Entorno aislado (*sandbox*):** Entorno habilitado por el manifiesto (**chrome2024sandbox**) con CSP relajada que autoriza la instanciación de trabajadores web (*Web Workers*) dedicados al cómputo intensivo de inferencia.

3.2. Estado del arte

3.2.1. Desarrollos en Argentina y la región

En el ámbito nacional, las iniciativas tecnológicas orientadas a la comunidad sorda se han enfocado predominantemente en la enseñanza y la consulta de vocabulario. El Diccionario de Señas del INJU opera como un índice léxico en línea sin capacidades de reconocimiento dinámico (INJU, 2024). La Confederación Argentina de Sordos (CAS) promueve herramientas pedagógicas bilingües de carácter estático (Confederación Argentina de Sordos, 2024). En cuanto a la animación digital, la aplicación móvil LSApp (Barán, 2018) introdujo un personaje tridimensional para reproducir entradas del diccionario y deletreo manual de manera unilateral.

En el plano asistencial comercial, la plataforma Blessit (Blessit, 2024) y servicios públicos como la Línea 144 (Ministerio de Justicia de la Nación, 2024) ofrecen teleinterpretación mediante videollamada con operadores humanos. Estos esquemas dependen de la disponibilidad de personal, generan demoras de conexión y condicionan la privacidad de los usuarios.

En el ámbito académico, el corpus LSA64 (Ronchetti et al., 2016) constituye el principal antecedente en reconocimiento automático local; no obstante, su captura controlada mediante guantes de colores restringe su aplicación directa sobre cámaras web convencionales, limitación observada en desarrollos posteriores basados en redes recurrentes (Mindlin, 2021).

3.2.2. Sistemas internacionales de reconocimiento y síntesis

A nivel internacional, el sistema SignAll (SignAll Technologies, 2026) traduce la lengua de señas americana (ASL) hacia inglés escrito mediante visión computacional y modelos lingüísticos. Sin embargo, su enfoque atiende la comprensión del oyente y omite la devolución gestual accesible hacia el usuario sordo. Por su parte, corpus a gran escala como WLASL (Li et al., 2020) han impulsado el reconocimiento léxico en video de ASL, aunque sin abordar la traducción de oraciones continuas ni la LSA.

En la dirección inversa (texto a señas), la empresa Hand Talk (Hand Talk, 2025) desarrolla avatares 3D para la traducción de texto hacia lengua de señas brasilera (Libras) y ASL. De igual forma, la fundación HETAH (HETAH, 2024) en Colombia ejecuta síntesis gestual a partir de oraciones en español. Sin embargo, estas plataformas no integran reconocimiento continuo de LSA, traducción semántica contextual y síntesis gestual interactiva dentro del flujo de una videollamada bajo un esquema híbrido.

3.3. Metodología de desarrollo

El proyecto se estructuró bajo un modelo de ciclo de vida iterativo e incremental (Pressman & Maxim, 2014), compuesto por 6 fases principales:

1. **Curación de datos y formalización léxica:** Delimitación de 100 señas de LSA en colaboración con un informante calificado; diseño del entorno de grabación; captura de clips en resolución 1080p; y generación del dataset de coordenadas vectoriales normalizadas.

2. **Entrenamiento y diagnóstico del clasificador gestual:** Experimentación de arquitecturas temporales (evaluación en ventanas de 8 a 32 fotogramas); optimización de hiperparámetros mediante marcos automatizados (Akiba et al., 2019); y desarrollo de métricas de diagnóstico mediante análisis AUC de Mann-Whitney.

3. **Desarrollo de los módulos semánticos bidireccionales y diseño de despliegue:** Ingeniería de prompts; ajuste fino con LoRA sobre modelos Qwen2.5 (Alibaba Cloud, 2024), SmolLM2 (Allal et al., 2024) y Llama 3.2 (Dubey et al., 2024) (traducción directa e inversa *Topic-Comment* (Massone & Martínez, 2012)); cuantización a binarios GGUF (Gerganov, 2023); evaluación mediante métricas BLEU, ROUGE-L y METEOR; y diseño conceptual de la arquitectura en la nube.

4. **Integración y renderizado gestual:** Acoplamiento del pipeline en el motor local ILSA mediante patrones de diseño desacoplados; comunicación con el servidor semántico; e intercepción de video en Google Meet mediante Canvas sintético y extensión Manifest V3 (Google Chrome Developers, 2024).

5. **Avatar e integración visual inversa:** Desarrollo del controlador del avatar para la representación esquelética de las glosas del canal inverso; implementación del subsistema de contingencia para deletreo manual dactilológico; e integración en la extensión.

6. **Fase de pruebas:** Medición de latencia y evaluación de la robustez del prototipo en videollamada.

3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial

Herramientas empleadas. Durante las fases de desarrollo y redacción se utilizaron dos herramientas auxiliares:

- **Cursor:** Entorno de desarrollo integrado asistido por inteligencia artificial, utilizado para la navegación del repositorio, autocompletado sintáctico y asistencia en la edición de código en Python y JavaScript.

- **Gemini (Google):** Modelo de lenguaje utilizado como soporte de consulta técnica, facilitador en la asimilación de conceptos teóricos y asistencia en la revisión del documento.

Alcance y propósito de su utilización. La intervención de dichas herramientas fue con el propósito de:

- **Consultas técnicas y conceptuales:** Asistencia en la comprensión de aspectos específicos relativos a la cuantización GGUF Q4_K_M con `llama.cpp`, fundamentos de LoRA y esquemas de comunicación asíncrona en extensiones Manifest V3.

- **Asistencia en el desarrollo de software:** Sugerencias de refactorización de código y resolución de incompatibilidades sintácticas en el entorno Chrome Extensions.

- **Revisión de formato:** Corrección gramatical y adecuación de estilo académico del informe.

Límites y responsabilidad de autoría. Ninguna herramienta generativa reemplazó la labor intelectual, analítica ni experimental del proyecto. La concepción de la arquitectura híbrida, la recolección y curación del corpus de 100 señas, el diseño e iteración de las redes neuronales, el ajuste de los modelos de lenguaje y el análisis de resultados constituyen desarrollo exclusivo de los autores, quienes asumen la responsabilidad técnica y académica por la totalidad del contenido.

3.4. Herramientas y tecnologías empleadas

Cuadro 3.1: Conjunto de tecnologías del sistema integrado

Categoría	**Herramienta**	**Rol en el sistema**
Entorno y Lenguajes	Python 3.11, JavaScript	Lógica de backend, motor local, servidor semántico y extensión web.
Visión Computacional	MediaPipe Holistic, OpenCV	Extracción de coordenadas articulares corporales y manipulación de video.
Aprendizaje Profundo	PyTorch 2.6	Inferencia local del clasificador gestual.
Modelos de Lenguaje	Transformers, PEFT, Unsloth	Fine-tuning eficiente con adaptadores LoRA para traducción directa e inversa.
Inferencia de Lenguaje	`llama.cpp`, GGUF	Inferencia del traductor en CPU.
Comunicación y Backend	FastAPI, Uvicorn	Servidor de interfaz local (ILSA) y microservicio semántico desacoplado.
Servicio en la nube	Google Cloud Run	Instancia del servicio semántico empleada en las pruebas experimentales.
Integración Web	Chrome Extensions (MV3)	Intercepción de media mediante Canvas `captureStream()` en Google Meet.
Transcripción	Vosk	Transcripción local de audio a texto en el cliente.
Síntesis de Salida	Síntesis del navegador y avatar	Generación de voz sintética y dibujo esquelético del avatar gestual.
Avatar	PoseFormat	Conversión de metadatos y almacenamiento en formato binario `.pose`.

Capítulo 4

Análisis de requerimientos

4.1. Requerimientos funcionales

- **RF-01 — Captura y extracción local de landmarks:** Capturar el video del usuario señante en el cliente y extraer los 33 puntos de pose y los 21 de cada mano, en tres coordenadas. Cada punto resta el centro de los hombros y queda dividido por la distancia entre hombros.
- **RF-02 — Clasificación gestual temporal local:** Clasificar localmente secuencias fijas de coordenadas en una de las 100 clases de LSA soportadas y utilizar arquitecturas de aprendizaje profundo para reportar la glosa asignada y su confianza.
- **RF-03 — Traducción semántica directa remota (LSA → Español):** Enviar a la nube el texto de glosas de un enunciado y recibir una oración en español rioplatense. Una glosa léxica sola, un deletreo de letras o una seguidilla de dígitos se resuelven en ILSA y no salen a la nube.
- **RF-04 — Emisión hacia el oyente:** Mostrar la traducción como subtítulo dentro del video, leerla en voz alta en la misma computadora, o hacer ambas cosas. El audio no reemplaza la pista de la llamada.
- **RF-05 — Captura del turno del interlocutor oyente:** Capturar la voz o texto del oyente en la videollamada durante su turno de intervención con el fin de ser procesada para su uso en el canal de retorno bidireccional.
- **RF-06 — Traducción semántica inversa remota (Español → LSA):** Transmitir el texto en español emitido por el oyente hacia el servicio semántico en la nube y obtener una secuencia ordenada de glosas bajo la estructura gramatical *Topic-Comment* canónica de la LSA.
- **RF-07 — Representación visual mediante un avatar:** Recibir las glosas generadas por el canal inverso y renderizar localmente en pantalla la animación de la seña correspondiente, y ejecutar deletreo manual dactilológico para términos fuera del léxico.
- **RF-08 — Integración con Google Meet:** Inyectar el video procesado y los canales de comunicación dentro de la pestaña de videollamada.

4.2. Requerimientos no funcionales

- **RNF-01 — Latencia y tiempo de respuesta:** La pausa que cierra el enunciado vale 4 segundos por defecto. La inferencia semántica remota y retorno al cliente no debe superar los 2 segundos bajo conexiones de banda ancha estándar.
- **RNF-02 — Privacidad de imagen y video:** Ningún flujo de video crudo o imagen de la cámara web debe ser transmitido hacia servidores externos o servicios en la nube; el intercambio de red con el backend se restringe exclusivamente a cargas textuales livianas (glosas o texto transcripto).
- **RNF-03 — Robustez en reconocimiento en vivo:** El clasificador debe superar el 75 % de acierto en luz doméstica. Ese valor es una meta de reconocimiento. No es el umbral de confianza: ILSA descarta la glosa cuando la confianza queda bajo 0,75.
- **RNF-04 — Desacoplamiento y escalabilidad de los servicios en la nube:** La arquitectura del backend semántico y del servidor de archivos .pose deben estar concebidas para operar de forma desacoplada y elástica, lo que permita sus despliegues en infraestructuras con auto-escalado horizontal a través de interfaces REST tipadas.
- **RNF-05 — Usabilidad y no invasividad:** La interfaz sobre la videollamada debe ofrecer indicadores visuales de estado que no ocluyan la visibilidad del interlocutor para mitigar la brecha comunicacional receptiva y expresiva.

4.3. Restricciones

- **RC-01 — Alcance léxico:** El vocabulario reconocido por el clasificador está acotado a 100 señas validadas por profesionales de LSA, enmarcadas en las pautas pedagógicas y lingüísticas vigentes.
- **RC-02 — Despliegue en la nube del prototipo:** Las pruebas no despliegan la arquitectura elástica del servicio semántico o del servicio de archivos .pose. La validación usa Cloud Run, de Google Cloud.
- **RC-03 — Compatibilidad de plataforma cliente:** El empaquetado del motor local (ILSA) y la extension estan diseñados para el sistema operativo Windows y navegadores basados en Chromium bajo las especificaciones de seguridad vigentes.
- **RC-04 — Dependencia de visión:** El clasificador depende del correcto funcionamiento de MediaPipe Holistic para la extraccion de landmarks.
- **RC-05 — Recursos del cliente objetivo:** El componente local del prototipo debe ser capaz de operar en laptops estándar (procesadores de 4 núcleos y 8 GB de memoria RAM, sin placa gráfica dedicada).

4.3.1. Especificación del entorno de ejecución cliente

A fin de asegurar la accesibilidad y democratización del sistema, se definió un hardware objetivo representativo del equipamiento informático estándar presente en ámbitos educativos y laborales:

Cuadro 4.1: Requisitos de hardware y software del dispositivo cliente

Componente	Requisitos mínimos (Dispositivo base)	Requisitos recomendados
Procesador (CPU)	Intel Core i3 (10ª gen.) / AMD Ryzen 3 (serie 3000) (4 núcleos / 4 hilos, base $\geq$ 2,0 GHz, soporte AVX2).	Intel Core i5 (11ª gen.+) / AMD Ryzen 5 (serie 4000+) (6 núcleos / 12 hilos).
Memoria RAM	8 GB DDR4 ($\geq$ 3,0 GB libres para el entorno de ejecución).	16 GB DDR4 / DDR5.
Gráficos (GPU)	Gráficos integrados (Intel UHD Graphics 620 / AMD Radeon Vega 3) con soporte WebGL.	Intel Iris Xe / AMD Radeon 600M+ o GPU dedicada básica.
Almacenamiento	Disco de estado sólido (SSD) con 2 GB de espacio libre.	SSD NVMe con $\geq$ 5 GB libres.
Cámara web	Resolución 720p (1280×720) a 24/30 FPS.	Resolución 1080p a 30 FPS.
Conectividad	Conexión de banda ancha ($\geq$ 10 Mbps bajada / $\geq$ 3 Mbps subida, latencia $<$ 80 ms).	Conexión por fibra óptica ($\geq$ 50 Mbps).
Entorno de software	Windows 10/11 (64 bits), Google Chrome 115+ (soporte Manifest V3).	Windows 11 (64 bits), Google Chrome última versión estable.

Criterio técnico de selección del procesador base La CPU base debe sostener tres cargas a la vez: la videollamada en el navegador, la extracción de puntos en WebAssembly y la clasificación de ILSA en CPU. Por eso el mínimo queda en cuatro núcleos, con instrucciones AVX2 para el modelo. Este informe no asigna una cantidad de núcleos a cada carga ni fija un tiempo de clasificación en milisegundos.

4.4. Casos de uso

4.4.1. Casos de uso esenciales en formato breve

- **CU-01 — Conversar con otra persona (Modo Sordo):** El usuario sordo seña frente a la cámara durante la videollamada. El sistema clasifica las señas en el equipo. Según el enunciado, arma el texto en ILSA o consulta el servicio semántico. La salida es subtítulo en el video, audio en esa computadora, o ambas.
- **CU-02 — Mano hábil:** El motor sabe espejar el vector si la persona es zurda. En la videollamada la sesión arranca con mano derecha y la extensión no ofrece el cambio. La herramienta de cámara del entrenamiento sí muestra los controles de diestro y de zurdo.
- **CU-03 — Configurar canal de salida:** En el panel de la extensión, el usuario señante elige la salida en Meet: Subtítulo, Audio o Ambos.

- **CU-04 — Traducir español a LSA mediante Avatar (Modo Oyente):** El usuario oyente habla o escribe durante la llamada; el sistema solicita la estructuración remota en glosas LSA y el avatar reproduce localmente las señas o las deletrea visualmente.

4.4.2. Casos de uso expandidos

Identificador	**CU-01: Conversar con otra persona (Modo Sordo)**		
Actor principal	Usuario señante		
Precondiciones	El usuario debe estar unido a una llamada.		
Postcondiciones	El interlocutor lee el subtítulo en el video, o la voz suena en la computadora de quien seña, según la salida elegida.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Realiza las señas correspondientes a lo que quiere comunicar	
	2		El Sistema procesa la seña, extrae los landmarks, clasifica la seña y la carga en un buffer temporal.
	3	Si necesita otra seña, vuelve al paso 1. Si no, espera la pausa elegida: 2, 3 o 4 segundos. El valor inicial es 4.	
	4		Si el enunciado es una glosa léxica sola, un deletreo o dígitos, ILSA arma el texto en el equipo. En los demás casos envía las glosas al servicio semántico. Muestra el resultado como subtítulo, como audio local, o con ambas salidas.
Extensiones (flujos alternativos)	-		
Requisitos especiales	-		

Identificador	**CU-02: Mano hábil**		
Actor principal	Usuario señante		
Precondiciones	El motor local está en marcha. En Meet no hace falta elegir la mano: la sesión arranca diestra.		
Postcondiciones	En la herramienta de cámara, el vector queda espejado si la persona elige zurdo. En Meet el vector queda diestro.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	En la herramienta de cámara del entrenamiento, elige diestro o zurdo.	
	2		Si la elección es zurdo, espeja el eje horizontal y cruza las dos manos. El modelo se entrenó con mano derecha.
	3	En la extensión de Meet no aparece este control.	La sesión de la llamada usa mano derecha.
Extensiones (flujos alternativos)	-		
Requisitos especiales	-		

Identificador	**CU-03: Configurar canal de salida del oyente**		
Actor principal	Usuario señante		
Precondiciones	El usuario debe estar unido a una llamada.		
Postcondiciones	La salida de la interpretación en Meet queda en subtítulo, audio local o ambas.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Abre el panel de la extensión.	
	2		Muestra tres opciones bajo «Salida en Meet»: Subtítulo, Audio y Ambos.
	3	Elige una opción.	
	4		Aplica esa salida. El audio suena en la misma computadora y no entra en la pista de la llamada.
Extensiones (flujos alternativos)	-		
Requisitos especiales	-		

Identificador	**CU-04: Traducir español a LSA mediante Avatar (Modo Oyente)**		
Actor principal	Usuario no señante		
Precondiciones	El usuario está unido a una llamada y la extensión está en modo oyente.		
Postcondiciones	El video de la llamada muestra el esqueleto de las glosas, o el deletreo si falta un archivo.		
Escenario Principal de éxito (flujo básico)		**Usuario oyente**	**Sistema**
	1	Habla, o escribe en el chat.	
	2		Si la entrada es voz, la transcribe en la misma computadora. Si es chat, toma ese texto.
	3		Envía la oración al servicio semántico y recibe las glosas.
	4		Dibuja el clip de cada glosa sobre la cámara. Si una glosa no tiene archivo, dibuja sus letras una por una.
Extensiones (flujos alternativos)	-		
Requisitos especiales	-		

Capítulo 5

Diseño del Sistema

5.1. Arquitectura del sistema

El diseño adopta una arquitectura híbrida. La extracción de puntos y la clasificación gestual corren en el equipo del usuario. El módulo semántico corre como servicio remoto.

5.1.1. Justificación de la partición y modelo de privacidad

Se probó una primera configuración con las tres etapas en el cliente: extracción, clasificación y traducción semántica. La extracción y la clasificación conviven con la videollamada. La demanda de memoria RAM del modelo de lenguaje obliga a retirarlo del equipo del usuario y a publicarlo como servicio en la nube. Se utiliza un filtro para evitar repeticiones ilógicas, y hacia el servicio sale solo el texto del turno en curso, ya cerrado, en forma de glosas o de oración. El video y los puntos articulares permanecen en el equipo.

5.1.2. Flujo integral de datos entre componentes.

La dinámica operativa del sistema conecta el navegador, el motor local y dos servicios remotos. La Figura 5.1 fija esa partición. El video y los puntos articulares quedan en el equipo. Hacia la nube sale texto: glosas en el canal directo, u oración en el canal inverso. El servicio de poses no traduce. Devuelve el archivo de la glosa.

Para modelar la interacción temporal y la secuencia de mensajes se analizan a continuación los dos canales comunicacionales de manera independiente.

Dinámica temporal del Canal Señante → Oyente (Modo Sordo)

En la Figura 5.2 se modela el ciclo de vida de un enunciado en LSA.

Dinámica temporal del Canal Oyente → Señante (Modo Oyente)

En la Figura 5.3 se describe la secuencia temporal que transforma el mensaje oral o escrito del oyente en representaciones visuales comprensibles en LSA para el usuario sordo.

5.2. Diseño de contratos de API REST

La comunicación entre el cliente y los servicios se rige por contratos tipados en formato JSON.

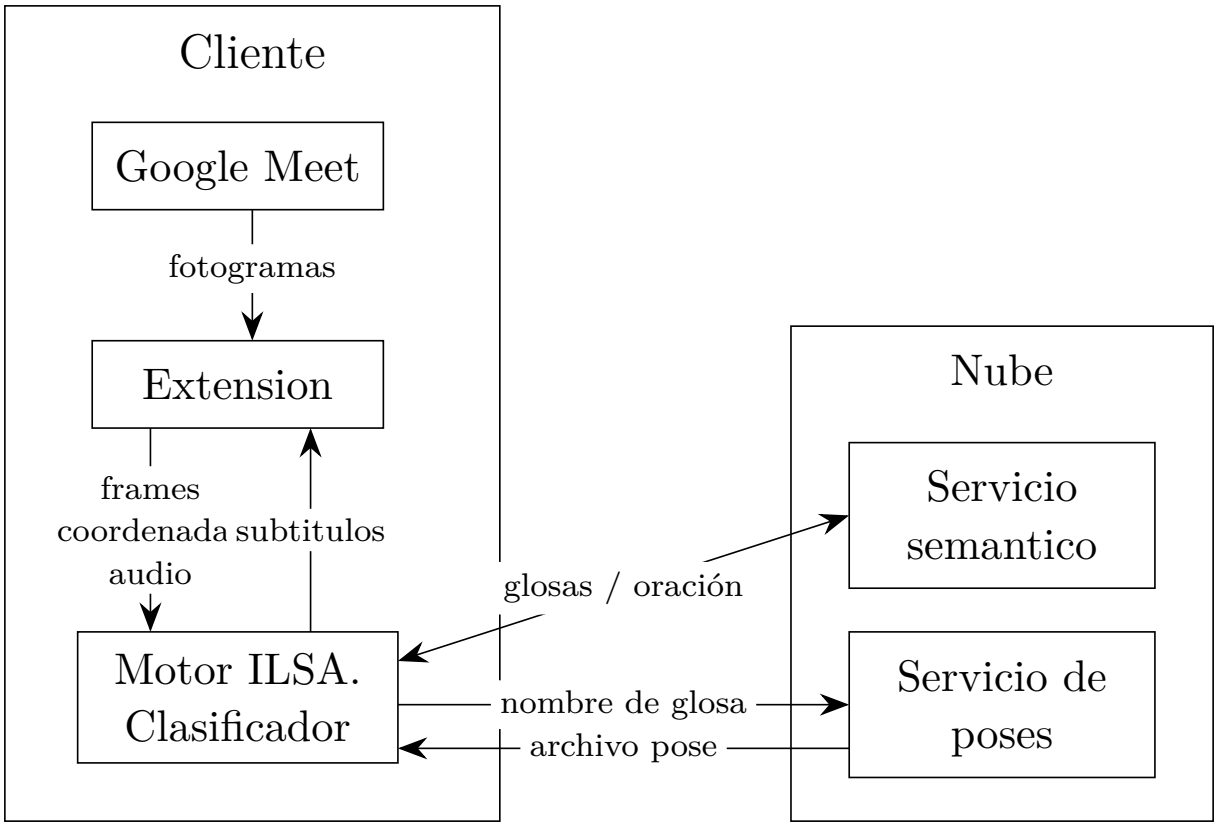


Figura 5.1: Diagrama de la arquitectura del sistema

5.2.1. API local del motor ILSA

El motor ILSA expone una interfaz HTTP en el bucle local, en `127.0.0.1:8765`. La extensión de Meet es el cliente y el proceso del motor es el servidor. El contrato cubre el modo señante, el modo oyente y dos recursos de archivo. La mayoría de las respuestas usa JSON. `GET /mediapipe/{name}` y `GET /download/exe` devuelven el archivo en binario.

Los métodos son `GET` y `POST`. Un cuerpo inválido, un modo incorrecto o un nombre de archivo no permitido producen `400 Bad Request`. Si el proceso todavía no tiene sesión, las rutas que consultan esa sesión responden `503 Service Unavailable`. La transcripción responde `502` cuando el reconocedor de voz falla. Un archivo ausente responde `404`. Las rutas de salud, de métricas, de transcripción, de MediaPipe y del ejecutable no exigen una sesión ya abierta.

Cuadro resumen de endpoints

El Cuadro 5.1 reúne todas las rutas del motor. El estado de las glosas reside en el búfer del proceso, no en la extensión.

`GET /health` indica si el proceso está en pie, en qué modo opera y si el clasificador y el traductor responden. `GET /state` y `POST /session` comparten la foto del motor: lateralidad, glosas pendientes, último español y estado del traductor. `POST /activity` solo mueve el reloj de pausa. `POST /utterance/end` cierra el enunciado aunque ese reloj no haya vencido.

En modo oyente, `POST /sign` responde 400. `POST /stt` transcribe el micrófono de la videollamada. `POST /hearing` traduce la oración y adjunta el clip. En modo señante,

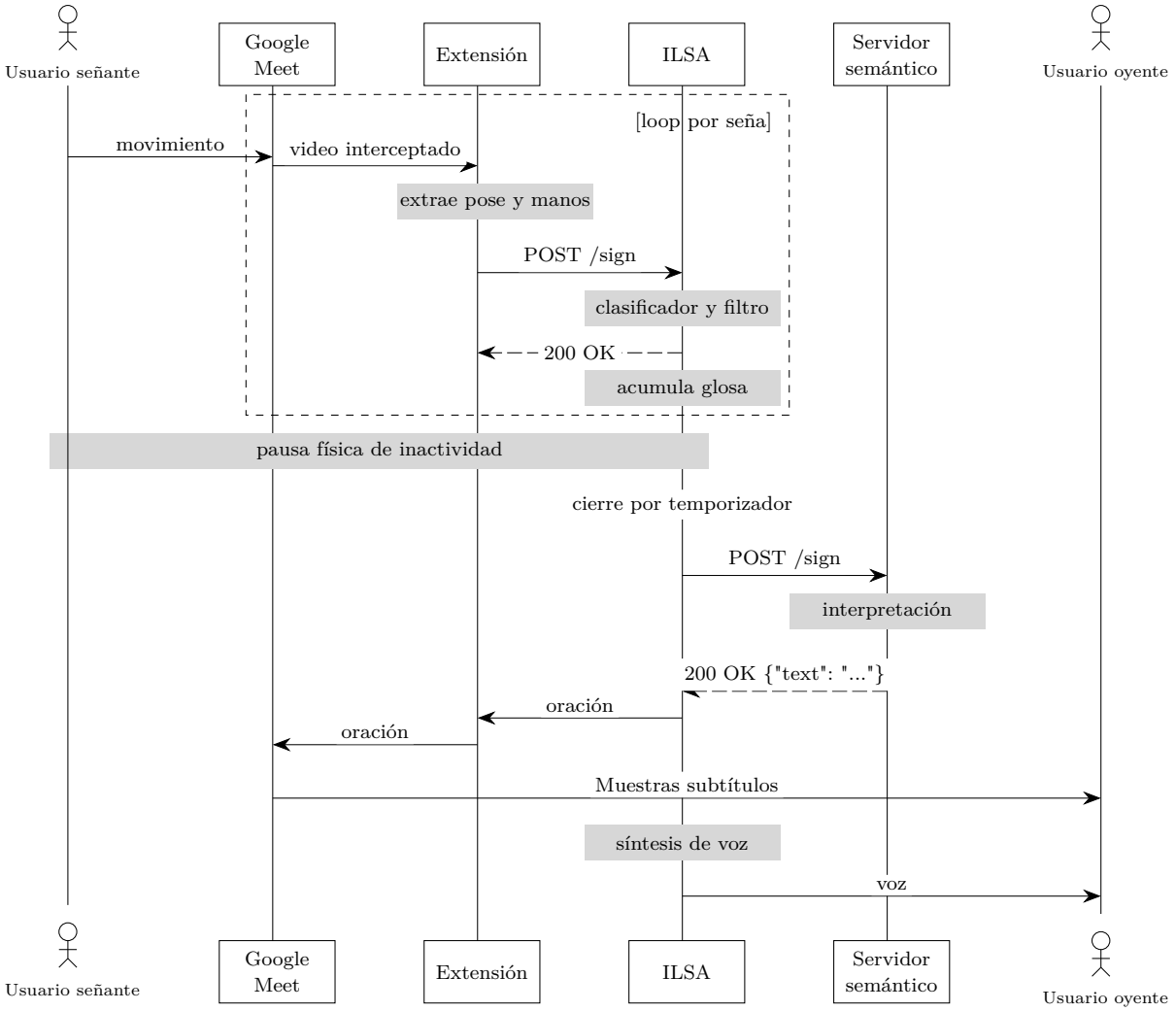


Figura 5.2: Diagrama de secuencia del flujo de traducción de Lengua de Señas a voz y subtítulos.

`POST /stt` y `POST /hearing` responden 400. De esas dos, solo `POST /hearing` consulta la sesión y puede responder 503.

Contrato POST `/session`

La apertura de sesión declara la lateralidad del señante. El valor `false` corresponde a ejecución diestra; `true` invierte el eje horizontal de los puntos de pose y manos antes de la clasificación. Una petición válida recibe `200 OK` y la foto del motor, la misma de `GET /state`, ya con la lateralidad aplicada.

Petición:

```
{
  "left_handed": false
}
```

Respuesta `200 OK` (campos principales):

```
{
```

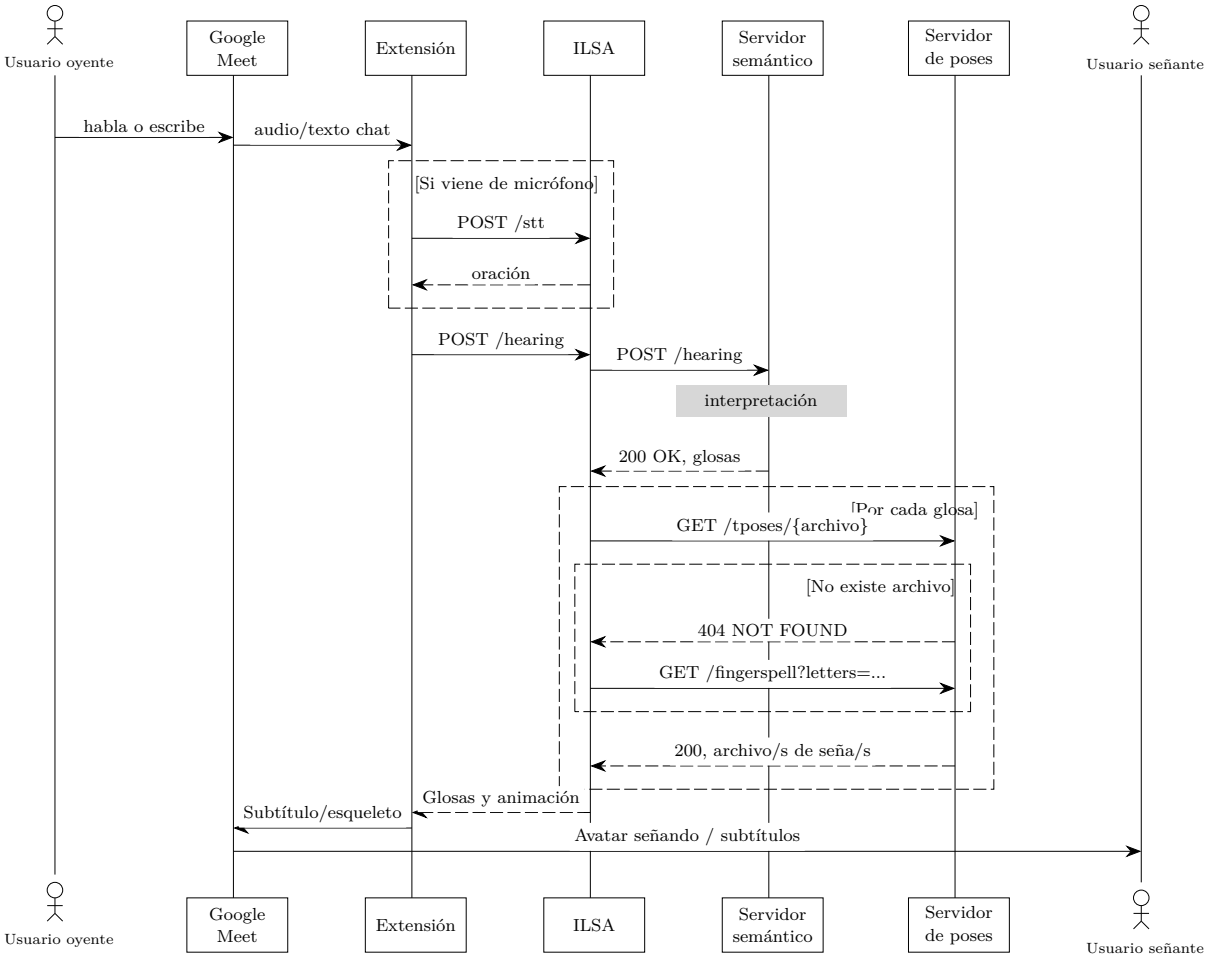


Figura 5.3: Diagrama de secuencia del flujo de interpretación de voz/texto a señas para el usuario señante.

```
  "left_handed": false,
  "classifier_ready": true,
  "glosses": [],
  "spanish": "",
  "semantic_ready": true
}
```

Contrato POST `/sign`

La ruta recibe una seña ya recortada, y solo en modo señante. El campo `frames` es una lista de objetos. Cada objeto trae la pose y las dos manos. El tope es 60 fotogramas. El motor suaviza esa lista y la deja en una matriz de 16×225 (33×3 de pose y 21×3 por mano). Un cuerpo vacío responde `400`. En modo oyente la misma ruta responde `400` y no clasifica.

Petición (secuencia abreviada):

```
{
  "frames": [
    {"pose": [], "left_hand": [], "right_hand": []}
```

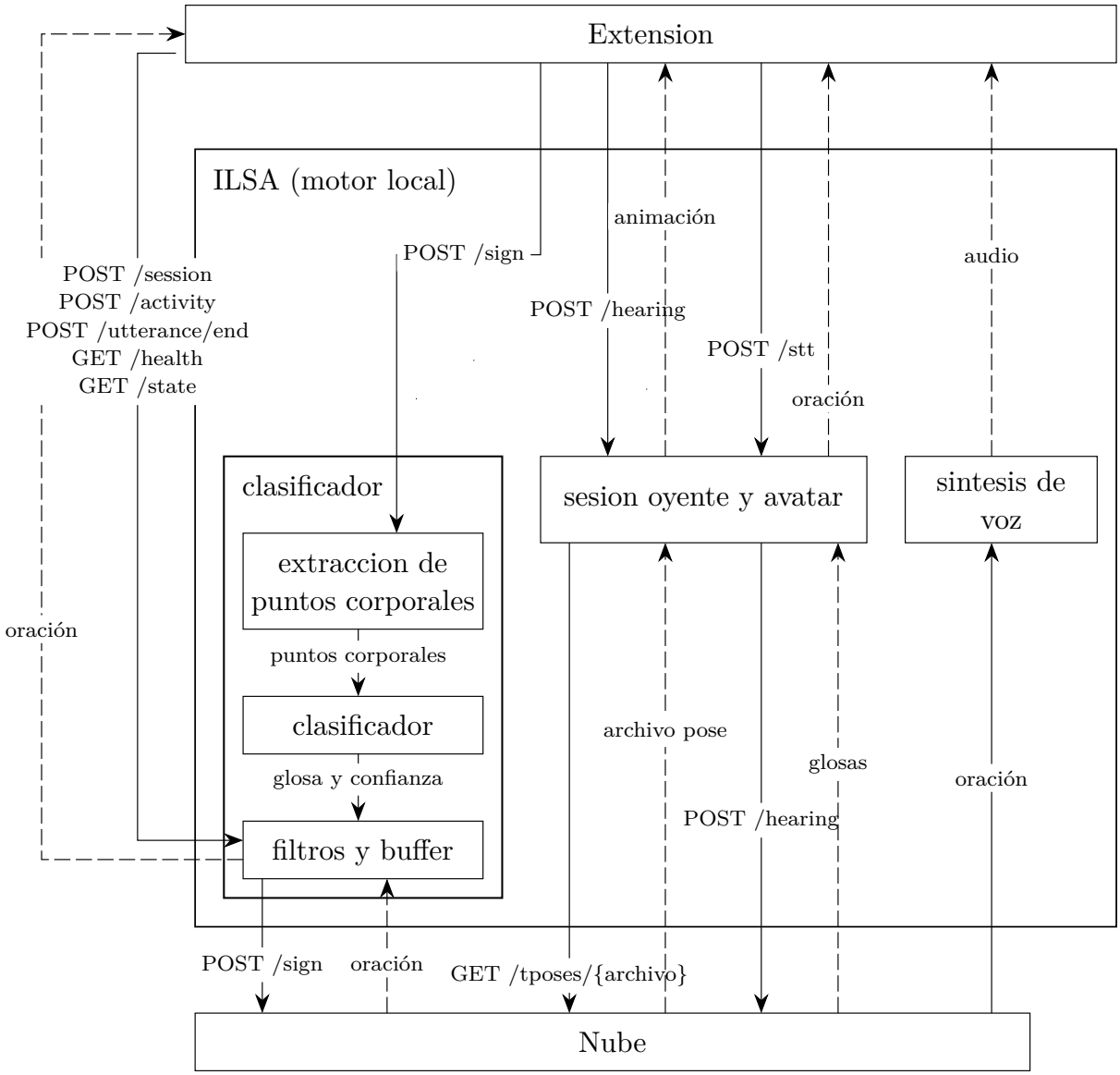


Figura 5.4: Arquitectura detallada del motor local ILSA y su interacción con los servicios en la nube.

```
  ],
  "client": {}
}
```

Una respuesta `200` puede aceptar o rechazar la seña. El rechazo por pausa entre inferencias, por seña demasiado corta o por una matriz inválida sigue en `200`, con `accepted` en falso y un motivo. No usa el código 400. Ese código queda para el cuerpo vacío y para el modo oyente.

```
{
  "accepted": true,
  "added": true,
  "gloss": "HOLA",
  "top3": [
    {"gloss": "HOLA", "confidence": 0.91}
```

Cuadro 5.1: Endpoints de la API local del motor ILSA (`http://127.0.0.1:8765`).

Método	Ruta	Contrato	Códigos
GET	`/health`	Disponibilidad del proceso, del modo, del clasificador y del traductor remoto. No altera estado.	200
GET	`/config`	Umbrales de captura del modo en curso.	200, 503
GET	`/state`	Búfer de glosas, último texto y estado del traductor.	200, 503
GET	`/metrics`	Conteos y tiempos de la sesión en curso.	200
POST	`/session`	Nueva sesión. El cuerpo fija la lateralidad con `left_handed`. La respuesta es la misma foto que `GET /state`.	200, 503
POST	`/sign`	Solo modo señante. Recibe hasta 60 fotogramas, cada uno con pose y manos. El motor los deja en una matriz de 16×225 y devuelve si la glosa entra al búfer.	200, 400, 503
POST	`/stt`	Solo modo oyente. Recibe audio WAV en Base64 y devuelve la oración en español.	200, 400, 502
POST	`/hearing`	Solo modo oyente. Recibe la oración y una marca de cierre. Consulta el canal inverso y, si el turno cerró, pide los archivos de pose.	200, 400, 503
POST	`/activity`	Sin cuerpo. Reinicia la pausa de unos 4 s y aplaza el cierre del enunciado.	200, 503
POST	`/utterance/end`	Cierra el enunciado del modo señante, envía las glosas al traductor y vacía el búfer.	200, 503
GET	`/mediapipe/{name}`	Archivo local de MediaPipe para la extensión. Admite `.wasm`, `.js`, `.data`, `.tflite` y `.binarypb`.	200, 400, 404
GET	`/download/exe`	Ejecutable del motor. Si no está en el equipo, entrega el archivo de arranque.	200, 404

```
  ],
  "glosses": ["HOLA"]
}
```

`added` indica si la glosa entró al búfer. Una repetición no admitida o una confianza baja dejan `accepted` en verdadero y `added` en falso. `top3` ordena hasta tres clases. El cierre del enunciado no viaja en esta respuesta: lo decide la pausa o `POST /utterance/end`.

5.2.2. API del servicio semántico remoto

La arquitectura local se comunica con el microservicio semántico en la nube mediante una API REST sobre HTTPS. Cada petición es independiente: el contrato no incluye memoria de turnos previos, es *serverless*. El canal directo corresponde a la traducción de glosas LSA a una oración en español rioplatense. El canal inverso corresponde a la traducción de una oración en español a una secuencia de glosas con orden *Topic-Comment*, apta para la animación del avatar.

El Cuadro 5.2 resume los endpoints del diseño.

Contrato de `POST /sordo` y `POST /translate` (canal directo)

`POST /sordo` y `POST /translate` comparten este contrato. La petición transporta las glosas reconocidas en el edge como una única cadena, con tokens en mayúsculas separados por espacios. La respuesta, en caso de éxito (código 200), incluye la oración en español, el identificador del SLM y el tiempo de inferencia en milisegundos (`ms`).

Petición:

```
{
  "glosses": "APELLIDO MIO PIÑON"
}
```

Respuesta (200 OK):

Cuadro 5.2: Endpoints del servicio semántico remoto.

Método	Ruta	Función
GET	`/health`	Consulta de disponibilidad: indica si los modelos de lenguaje (SLM) residen en memoria RAM y si el servicio admite inferencia.
POST	`/sordo`, `/translate`	Canal directo (LSA → español rioplatense). Las dos rutas reciben la misma secuencia de glosas y devuelven la misma oración.
POST	`/hearing`	Canal inverso (español → LSA Topic-Comment). Recibe el texto del interlocutor oyente y devuelve la secuencia de glosas. Esas glosas alimentan después el servicio de archivos del avatar.

```
{
  "spanish": "Mi apellido es Piñon.",
  "glosses": "APELLIDO MIO PIÑON",
  "model_used": "qwen2.5-1.5b.gguf",
  "ms": 412.3,
  "queued_ms": 0.0
}
```

Contrato de `POST /hearing` (canal inverso)

La petición transporta la oración en español rioplatense, ya transcripta en el edge. La respuesta, en caso de éxito (código 200), incluye la secuencia de glosas en una cadena de tokens en mayúsculas, el SLM empleado y el tiempo de inferencia.

Petición:

```
{
  "spanish": "Hola, ¿cómo estás?"
}
```

Respuesta (200 OK):

```
{
  "glosses": "HOLA VOS COMO",
  "spanish": "Hola, ¿cómo estás?",
  "model_used": "qwen2.5-1.5b.gguf",
  "ms": 380.1,
  "queued_ms": 0.0
}
```

El campo `queued_ms` registra la espera en cola si el SLM atiende otra inferencia; no forma parte del texto traducido. Un código 503 indica que el modelo no está listo en RAM. El endpoint `GET /health` permite al *edge* diferir el encendido del clasificador local.

5.2.3. API del servicio de archivos del avatar

El canal inverso no anima por sí solo. Un segundo servicio remoto entrega el archivo de movimiento de cada glosa. El nombre del archivo, sin la extensión, es la glosa ya normalizada. El puerto, la imagen y la plataforma de ese servicio quedan en la Sección 6.4.1.

El Cuadro 5.3 resume las rutas. La de salud responde JSON. La del archivo responde el binario, no un documento JSON.

Cuadro 5.3: Endpoints del servicio de archivos del avatar.

Método	Ruta	Función
GET	`/health`	Disponibilidad del proceso. Responde 200 con `{"status": "ok"}`.
GET	`/tposes/{glosa}.pose`	Archivo de esa glosa. El nombre debe coincidir de forma exacta, extensión incluida. 200 y cuerpo binario si existe. 404 si no existe.
GET	`/fingerspell`	Solo letras del alfabeto manual, varias en el parámetro `letters`, separadas por coma. Por ejemplo, `GET /fingerspell?letters=H,O,L,A`. 200 y el archivo de cada letra. 400 si el pedido viene vacío o trae algo que no es letra. 404 si falta el archivo de una de esas letras.

ILSA arma el nombre antes de la solicitud. El resultado es una sola cadena de letras mayúsculas y números, sin tildes, sin espacios y sin otros signos. Las reglas se aplican en este orden:

1. Toda la glosa pasa a mayúsculas.
2. Queda la forma masculina.
3. Las vocales con tilde pasan a la vocal simple. `CAFÉ` queda `CAFE`.
4. Se borran guiones bajos.
5. La letra `Ñ`, cuando esa es toda la glosa, se pide como `ENIE`. Una `Ñ` dentro de la palabra pasa a `N`. `ESPAÑA` queda `ESPANA`.
6. La glosa de parentesco `PAPA` se pide como `PADRE`. El archivo `PAPA.pose` corresponde al alimento. Para el parentesco, ILSA usa `PADRE`.

La dirección del servicio y la caché local se describen en la Sección 6.4.1. La búsqueda de una glosa compuesta queda en la Sección 5.3.

Ausencia de un archivo

Cuando el nombre pedido no tiene archivo, el servicio puede cerrar ese hueco de cuatro maneras. Las cuatro se consideraron. El prototipo adopta la cuarta.

Letra por letra. Un archivo ausente responde 404. El cliente pide después cada letra del deletreo en una solicitud propia. A favor, cada letra queda como un archivo independiente, y una letra que falta no anula a las demás. En contra, un nombre largo exige un viaje de red por letra, después del 404 inicial.

Lote de letras o de glosas. El archivo ausente sigue en 404. Otra solicitud admite varias letras juntas, y también todas las glosas del enunciado. A favor, el deletreo, o el enunciado entero, viaja en un solo pedido. En contra, la misma solicitud mezcla la seña léxica con la letra. Si falta un nombre en el medio, la respuesta deja de ser un archivo o un 404 y tiene que explicar un éxito parcial.

Deletreo en la misma ruta. La ruta que pide la glosa, si no hay archivo, responde con las letras que forman ese nombre. A favor, el cliente no hace un segundo pedido: el deletreo vuelve en la misma llamada. En contra, una misma ruta a veces entrega una seña y a veces una lista de letras. El 404, que avisa que el diccionario no tiene esa glosa, desaparece. Una caché bajo ese nombre guardaría el deletreo en el lugar de una seña que todavía no existe.

Ruta exclusiva de deletreo. El archivo ausente responde 404. El deletreo tiene su propia ruta, `GET /fingerspell`, y esa ruta solo acepta letras. Varias letras entran en una sola solicitud, como en `GET /fingerspell?letters=H,O,L,A`. A favor, la ruta del archivo conserva un nombre y un archivo, o un 404. La ruta del deletreo no acepta una glosa léxica, y un nombre no cuesta un viaje por letra. En contra, después del 404 el cliente hace una segunda solicitud, la del deletreo.

La cuarta opción es la retenida. El 404 conserva un significado único: esa glosa no tiene seña. El cliente, y no el servicio, decide si ese nombre se deletrea. La primera opción conserva el mismo 404, pero multiplica los viajes en cada nombre propio. La segunda ahorra viajes, y a la vez abre la puerta a pedir todo el enunciado de una vez: un hueco en el medio obliga entonces a un contrato de éxito parcial. La tercera ahorra el segundo viaje, y esconde la ausencia dentro de una respuesta exitosa: quien llama ya no distingue la falta de seña de la seña misma. La ruta aparte deja el archivo como lectura exacta, apta para la caché y para las réplicas de solo lectura, y concentra el deletreo en un pedido que solo entiende letras. `CH` y `Ñ` cuentan como una letra, igual que en la Sección 5.3.4.

5.3. Diseño de la representación gestual inversa

El canal oyente recibe del traductor inverso una lista de glosas en mayúsculas. Esa lista, por sí sola, es texto: la persona sorda todavía no ve la seña. El diseño convierte cada glosa en el movimiento correspondiente, tomado de un archivo que ya contiene la seña medida sobre un video de referencia. El archivo no guarda una malla ni las rotaciones de un muñeco. Guarda, fotograma a fotograma, la posición de puntos articulares (*landmarks*): hombros, codos, muñecas, las dos manos y un contorno de cara. El servicio de la Sección 5.2.3 entrega ese archivo. Lo que sigue fija cómo se prepara antes de mostrarlo, cómo se une una seña con la siguiente y qué ocurre cuando la glosa no tiene archivo. La tasa común, el peso del suavizado, el tamaño del lienzo, el umbral de confianza, la longitud del puente y la duración de la entrada y de la salida quedan en la Sección 6.4.4.

5.3.1. Clip de referencia

El archivo de cada glosa funciona como el negativo de la seña. Para cada fotograma del video de origen conserva tres cosas: la posición de los puntos en píxeles de esa imagen, la cantidad de fotogramas por segundo y una confianza entre 0 y 1. La confianza dice cuán seguro está el estimador de haber visto ese punto. Esos datos quedan a la resolución del video original, sin achicarlos de antemano.

La escala al tamaño del lienzo se aplica en el momento de dibujar, y no dentro del archivo. El clasificador, en cambio, guarda las coordenadas como fracción del ancho y del alto, porque a la red le alcanza la forma del gesto. Si ese mismo número fraccionario se archivara para el avatar, el visor lo leería como una cantidad de píxeles: un valor 0,5 pasaría a significar medio píxel, y el cuerpo quedaría comprimido contra la esquina del cuadro. El diccionario animable archiva por eso píxeles de imagen. La reducción a un tamaño común ocurre después, cuando ya se conoce el recuadro en el que la seña se va a mostrar.

El archivo crudo tampoco se recorta. Un video de diccionario muestra a la persona desde antes de la seña: levanta la mano, realiza el gesto y vuelve a una postura de reposo. El significado está en el tramo del medio, que aquí se llama núcleo. El levantamiento inicial es la preparación y el regreso al reposo es la retracción. El instante en que empieza el núcleo y el instante en que termina se anotan en un índice aparte, con un tiempo de entrada y un tiempo de salida por glosa. La reproducción lee ese índice. El negativo queda entero, de modo que un cambio del recorte no obliga a estimar el video otra vez.

Al dibujar no entran todos los puntos que el estimador produjo. MediaPipe Holistic marca 33 puntos de cuerpo, 468 de cara y 21 en cada mano. El diseño conserva el torso, de la cintura hacia arriba, y las dos manos completas. Las piernas se apagan, porque en este recuadro la seña se lee de medio cuerpo. Dentro de los 33 puntos de cuerpo también hay una cara gruesa y unas puntas de dedos muy simples. Esos puntos se apagan en el dibujo. Si se trazan junto con los 21 puntos de cada mano y con el contorno fino de la cara, el espectador ve una mano encima de la otra y una cara encima de la otra.

La posición del cuerpo expresada en metros, que el estimador entrega además de la posición en la imagen, se deja de lado por una razón distinta. Esa posición en metros no incluye las 21 articulaciones de la mano. Usarla como si fuera el esqueleto completo deja las manos pegadas al pecho aunque en el video estén junto a la cara. La lectura del avatar es frontal, y lo que importa es el lugar de cada punto en el plano de la imagen. La profundidad de cada mano, además, está medida respecto de su propia muñeca y en una escala mucho menor que el desplazamiento horizontal y vertical. Es un dato interno del estimador, y no una distancia en metros hasta la cámara.

5.3.2. Preparación y empalme

Dos archivos recién descargados todavía no pueden reproducirse uno detrás del otro. Cada video trae su propia velocidad, su propio encuadre y sus propios huecos allí donde el estimador perdió una mano. El diseño obliga a que todo clip atraviese la misma preparación, siempre en el mismo orden. El orden importa porque cada paso deja el material en condiciones para el siguiente: un hueco que no se rellena antes del empalme se convierte, en el empalme, en un salto.

Cadencia común. Los videos de origen no comparten necesariamente la misma cantidad de fotogramas por segundo. Si un clip trae 24 y el siguiente trae 30, una espera de seis

fotogramas dura más en el primero que en el segundo, y el empalme pierde la noción de tiempo. Antes de cualquier otro retoque, todos los clips se llevan a una misma tasa. A partir de ahí, un fotograma significa la misma duración en todas las señas.

Recorte del núcleo. La reproducción usa el índice de entrada y de salida, y deja fuera la preparación y la retracción. Si se mostrara el video completo, quien mira vería a la persona llegar a la seña y retirarse de ella, y el final de una glosa se confundiría con el comienzo de la otra. El núcleo, que es el tramo donde la configuración y el movimiento distinguen esa seña de las demás, es lo único que pasa a la secuencia visible.

Relleno de huecos breves. Cuando el estimador deja de ver una mano durante unos pocos fotogramas, suele escribir el punto en el origen, es decir en la coordenada $(0, 0)$, o dejar un vacío entre dos fotogramas válidos. Si ese vacío es corto y está rodeado de fotogramas en los que la mano sí se vio, el diseño completa la posición con los vecinos válidos. El hueco largo, en el que la mano realmente no participa de la seña, no se inventa: lo resuelve el paso siguiente.

Mano en espera. Hay señas de una sola mano. En esos clips, el estimador puede copiar la mano visible sobre el lugar de la mano ausente, de modo que las dos manos quedan casi en el mismo sitio, o puede dejar la mano ausente en el origen. El diseño distingue esos dos casos de una mano que sí participa. Si las dos manos coinciden en una seña de una sola mano, la copia se retira. La mano que no participa se estaciona al costado del torso, y el antebrazo se dibuja como una línea continua desde el codo hasta la muñeca, para que la muñeca no aparezca quebrada en un ángulo que la seña no tiene. Esa postura de costado es la espera de la mano, y es distinta de la espera de todo el cuerpo, que se describe más abajo.

Suavizado. Aunque la mano esté bien seguida, la posición estimada tiembla de un fotograma al siguiente. El diseño mezcla cada medición nueva con la pose ya aceptada en el fotograma anterior, por un promedio exponencial. Un peso alto sigue al estimador con mayor fidelidad y deja pasar el temblor. Un peso bajo alisa de más y retrasa el gesto respecto del núcleo. El peso es el mismo para todo el diccionario. Su valor, y la ecuación, están en la Sección 6.4.4.

Lienzo común. Los videos no comparten resolución ni tamaño de cuerpo. Si cada clip se dibujara a su escala original, al pasar de una glosa a la otra el esqueleto cambiaría de tamaño en medio de la frase. Después del suavizado, el cuerpo se lleva a un recuadro común y se escala hasta un mismo ancho de hombros. Dos señantes de distinta talla, y dos tomas de distinto encuadre, ocupan entonces el mismo espacio, y el empalme no produce un salto de escala.

Recién después de esos seis pasos el clip está en condiciones de unirse a otro. El dato que más daño hace, si se le deja llegar, es la coordenada nula. Supóngase que el último fotograma útil de `HOLA` tiene la muñeca en un punto interior del cuadro, y que el primero de `MAMÁ` trae esa misma muñeca en $(0, 0)$ porque el estimador no la vio. Una interpolación de unos pocos fotogramas entre esos dos puntos dibuja la mano en viaje hacia la esquina, como si la seña la llevara hasta allí. El relleno y la mano en espera existen para que el empalme reciba solo coordenadas válidas. Entre el último fotograma del núcleo de una

glosa y el primero del núcleo de la siguiente, el diseño inserta un puente: una serie breve de fotogramas nuevos, que no estaban en ninguno de los dos archivos. La serie es más larga cuando las muñecas de un extremo y del otro quedan más lejos, para que el recorrido no sea un salto. La curva con la que se reparte ese recorrido se llama *smoothstep*: el punto sale despacio, acelera en la mitad y llega despacio, de modo que la unión no arranca con un tirón. Cada punto viaja en línea recta, en el plano de la imagen, desde donde está hasta donde tiene que estar. El archivo no contiene ángulos de huesos, y por eso el puente no gira una articulación: traslada los puntos.

Ese puente es un recurso gráfico. En la lengua, la coarticulación es otra cosa: el final de una seña ya anticipa la configuración de la siguiente. El puente no modela ese anticipo. Toma la pose final de un núcleo ya recortado y la pose inicial del núcleo siguiente, y cubre el espacio que hay entre ambas para que el ojo no vea un corte. El interior de cada seña llega al empalme tal como quedó después de la preparación. La preparación de un clip tiene un costo de cálculo, y la descarga del archivo tiene un costo de red. Una vez que un clip quedó preparado, el diseño lo conserva en una caché en memoria. Si la misma glosa vuelve a aparecer en el enunciado, o en un enunciado posterior de la misma sesión, se reutiliza esa copia: no se vuelve a preparar y no se vuelve a pedir por la red. Además, la secuencia visible tiene un techo de 24 glosas y de 1200 fotogramas. Por encima de ese techo, un texto largo ocuparía el turno de la llamada con una animación que no termina. El diseño corta la secuencia en el techo. El Cuadro 5.4 resume estas previsiones. El detalle de cada una está en los párrafos anteriores.

Cuadro 5.4: Previsiones del diseño frente a fallos del estimador y del léxico.

Situación	**Previsión**
La mano desaparece durante unos pocos fotogramas y el estimador escribe $(0, 0)$.	Completa el hueco con los fotogramas vecinos en los que la mano sí se vio.
En una seña de una sola mano, las dos manos quedan en el mismo lugar.	Retira la copia y estaciona ese brazo al costado del torso.
La confianza de un punto es demasiado baja.	Omite ese punto en el trazo.
La posición estimada tiembla de un fotograma al otro.	Mezcla la medición nueva con la pose anterior, con un peso fijo.
La muñeca final de una glosa queda lejos de la muñeca inicial de la siguiente.	Inserta un puente breve de fotogramas nuevos entre los dos núcleos, sin alterar el interior de cada seña.
La glosa no tiene archivo.	La deletrea con los clips del alfabeto manual.
Tampoco están los archivos de las letras.	Deja el esqueleto en la postura de espera y avisa en la interfaz.
El enunciado supera 24 glosas o 1200 fotogramas.	Corta la secuencia en ese techo.

5.3.3. Presentación en la videollamada

Quien está del otro lado de la llamada ve la imagen que el navegador entrega como cámara. El diseño sustituye esa imagen por un lienzo partido en dos mitades de la misma altura. La mitad izquierda muestra la cámara de la persona, recortada para llenar su mitad sin estirar la figura. La mitad derecha muestra el esqueleto. Las dos mitades comparten la altura para que el intérprete y la persona queden a la misma escala.

Mientras no llega una lista de glosas, el esqueleto permanece en espera y no adelanta ninguna seña. La postura de espera se toma de un tramo en el que la persona todavía no empezó el gesto, y se escala al mismo ancho de hombros del lienzo. El antebrazo queda casi horizontal, levemente hacia abajo, y la mano sigue esa línea. La cara de la espera es un óvalo con cejas, ojos, nariz y boca, suficiente para situar la cabeza respecto de las manos.

Cuando la lista de glosas ya está resuelta y los fotogramas están empalmados, la reproducción no avanza un fotograma del clip por cada imagen que publica la llamada. La llamada puede publicar 24 imágenes por segundo mientras el clip quedó preparado a otra tasa. Si se avanzara al ritmo de la llamada, la seña saldría más lenta o más rápida que el video de origen. El diseño mide el tiempo transcurrido y elige el fotograma del clip que corresponde a ese tiempo.

Esa reproducción tiene tres tramos. En la entrada, un tramo breve a la tasa del clip, la postura de espera se transforma en el primer fotograma de la seña, y el contorno de cara de la seña aparece de a poco. En el tramo central corre la secuencia ya preparada, con los puentes entre glosas ya incluidos en la lista de fotogramas. En la salida, de la misma duración que la entrada, el último fotograma vuelve a la espera, el contorno de la seña se apaga y queda la cara simple de la espera. La curva de la entrada y de la salida es la misma *smoothstep* del puente, de modo que el esqueleto no aparece ni desaparece de golpe. Al terminar, el clip se suelta y el esqueleto queda otra vez en espera, listo para la frase siguiente.

Hacia el lienzo no viajan los cientos de puntos del archivo. Viaja solo lo que hace falta para leer la seña en una mitad de cuadro: ocho puntos de pose, que son los dos hombros, los dos codos, las dos muñecas y las dos caderas; los 21 puntos de cada mano; y un contorno de cara formado por el óvalo, las cejas, los ojos, la nariz y los labios. En el recuadro pequeño de la llamada, esa malla se ve como una mancha densa y aporta poco a la configuración de los dedos, que es lo que la persona sorda necesita distinguir. Un punto de confianza baja se envía como ausente. El trazo lo saltea, en lugar de unir el punto anterior con una posición en la que el estimador casi no cree. Cada dedo lleva un color fijo, y ese color es el mismo en las dos manos: pulgar rojo, índice naranja, medio amarillo, anular verde y meñique azul. La palma es blanca. El color permite seguir un dedo cuando las manos se cruzan delante del cuerpo y, sin él, los trazos de una y otra mano se confundirían. La orientación también está fijada. En una toma de frente, la mano derecha de la persona aparece del lado izquierdo de la imagen, porque la cámara la ve de frente. El lienzo respeta esa orientación y no da vuelta el esqueleto para que la mano hábil quede del lado que el espectador llamaría derecho. La miniatura propia de la videollamada, en cambio, suele mostrarse espejada, como un reflejo. El diseño exige que esa persona y quien está del otro lado vean lo mismo: la cámara en la mitad izquierda y el esqueleto, sin invertir, en la mitad derecha. El mecanismo que deshace el espejo solo en la vista local queda en la Sección 6.4.4.

5.3.4. Contingencia léxica

El nombre con el que se pide cada archivo sigue las reglas de la Sección 5.2.3: mayúsculas, sin tildes y sin espacios. Aquí el problema es otro. Una lista de glosas puede nombrar una seña con una palabra o con varias, y también puede nombrar algo que el diccionario no tiene.

La búsqueda prueba primero el grupo más largo. Toma tres glosas seguidas y pregunta si existe un archivo para ese grupo. Si no existe, prueba las dos primeras. Si tampoco

existe, prueba una sola. El motivo es que una expresión como `BUENAS TARDES` puede ser una única seña, con un único archivo, y partirla de entrada en `BUENAS` y `TARDES` mostraría dos gestos distintos del que corresponde. Un dígito aislado, como el carácter `1`, se busca por el nombre de ese número, `UNO`, porque el diccionario nombra el número y no el signo con el que se escribe.

Cuando ni el grupo ni la palabra tienen archivo, el diseño deletrea la palabra con los clips del alfabeto manual. Así, un nombre propio como `FRANCISCO`, si no tiene seña propia, se muestra como la secuencia `F`, `R`, `A`, `N`, `C`, `I`, `S`, `C`, `O`. En ese alfabeto, `CH` es una sola letra, y no la letra `C` seguida de la letra `H`. `Ñ` también es una sola letra. Esas letras viajan juntas en `GET /fingerspell`, como fija la Sección 5.2.3. Esta regla pertenece al deletreo. Es independiente de la regla del nombre de archivo, que solo decide cómo se escribe la cadena en el pedido.

Si falta el diccionario entero, faltan también las letras. En ese caso no hay con qué deletrear. El esqueleto permanece en la postura de espera, y la interfaz informa que el diccionario no está disponible. La palabra no se omite sin aviso, y tampoco se reemplaza por un movimiento aproximado: lo único que el lienzo puede mostrar es un clip que existe.

5.4. Diseño de la extensión de videollamada

La extensión une la videollamada con el motor local. El requerimiento RF-08 pide esa unión dentro de la pestaña de la llamada. La Figura 5.5 fija el reparto de cada bloque. Su construcción queda en el Capítulo 6.

5.4.1. Frontera

La extensión es la única pieza que conoce Google Meet. No clasifica una seña, no arma la oración y no guarda el enunciado. Esas tareas pertenecen a ILSA. El búfer de glosas vive en el motor, como ya fija el Cuadro 5.1. Toda llamada de la extensión usa la dirección local de ese cuadro.

En modo señante el temporizador envía a `POST /sign` los puntos de la seña ya cerrada. ILSA devuelve la oración. Esa oración llega al lienzo como subtítulo. El estado y las glosas llegan al indicador.

En modo oyente el micrófono entrega el audio a `POST /stt` y recibe la oración transcrita. Esa oración, o el texto ya escrito en el chat, entra por `POST /hearing`. ILSA devuelve la animación y los subtítulos al lienzo, y las glosas al indicador. La extensión no llama al traductor ni al servicio de archivos. Esa llamada sale de ILSA.

5.4.2. Dos vistas de la misma llamada

Quien está del otro lado solo ve la imagen que la llamada transmite. En modo señante, esa imagen lleva el subtítulo de la oración. En modo oyente, lleva el lienzo partido de la Sección 5.3.3: la cámara en una mitad y el esqueleto en la otra, junto con los subtítulos.

Quien tiene la extensión ve, además, el indicador de la Figura 5.5. El indicador informa si el motor responde, cuál fue la última glosa y si el turno sigue abierto. Ese aviso no entra en la imagen transmitida. Así no tapa al interlocutor y se cumple el requerimiento RNF-05.

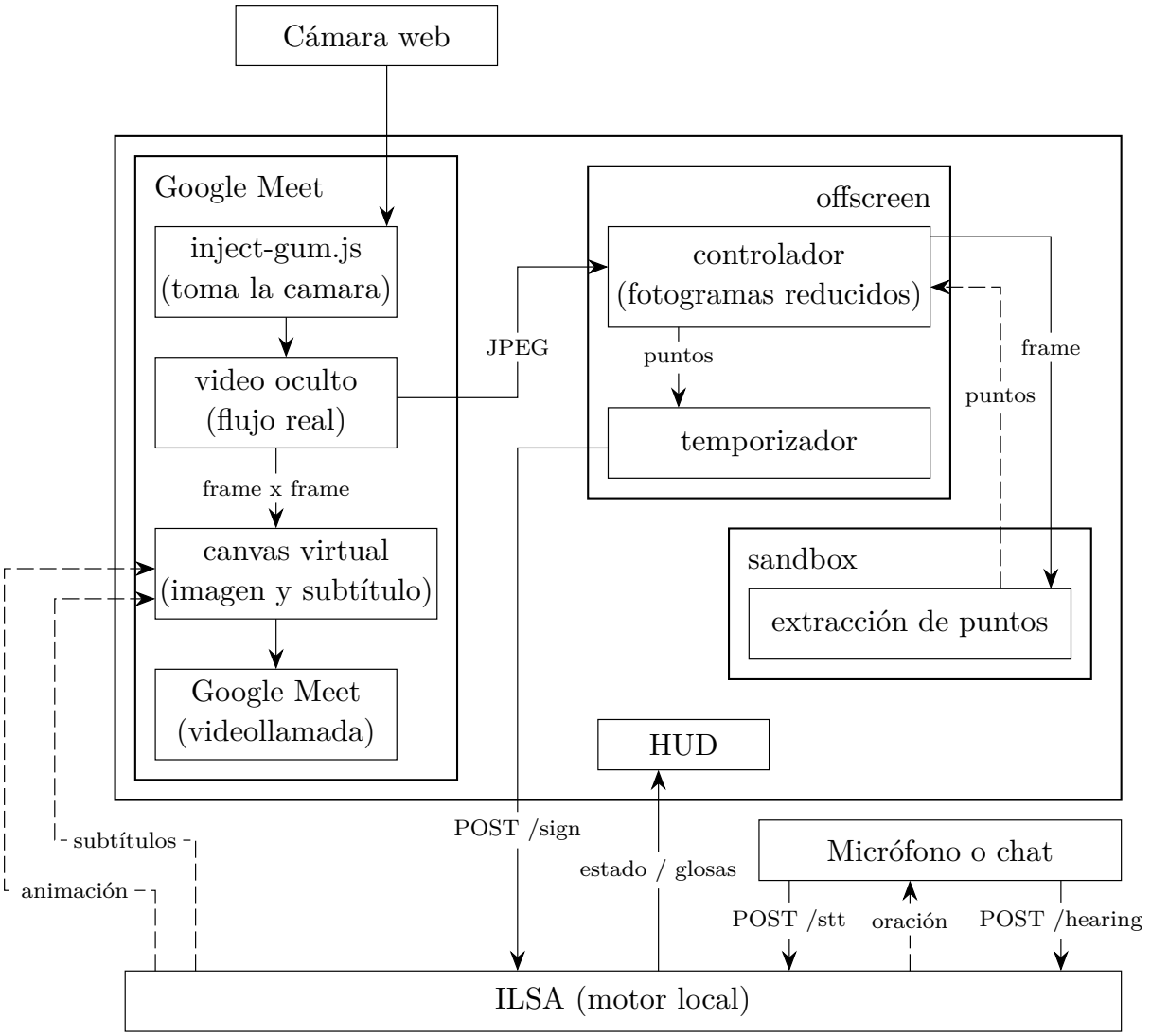


Figura 5.5: Diagrama de arquitectura del flujo de video, procesamiento offscreen e integración con el motor ILSA.

5.4.3. Aislamiento

Dentro del navegador el diseño separa tres ámbitos. La Figura 5.5 los dibuja como tres bloques.

El ámbito de la página es el bloque Google Meet. La toma de la cámara recibe la cámara web. El video oculto conserva el flujo real. El lienzo recibe esa imagen cuadro a cuadro, y la llamada publica el lienzo. Este ámbito es el único que toca la videollamada.

El ámbito de coordinación agrupa al controlador y al temporizador. El controlador recibe del video oculto un JPEG ya reducido, lo entrega a la visión y recibe los puntos. Esos puntos pasan al temporizador. En modo señante, el temporizador habla con ILSA por `POST /sign`.

El ámbito de visión extrae los puntos. Recibe un fotograma y devuelve los 225 puntos de pose y manos. No conoce Meet ni la dirección del motor.

El micrófono o el chat quedan fuera de esos tres bloques. En modo oyente hablan con ILSA. Ni el controlador ni la visión entran en ese camino.

Entre los tres ámbitos no circula el video crudo. Hacia la red salen solo glosas u oraciones, y salen desde ILSA. Los puntos articulares permanecen en el equipo. Esa partición sostiene el requerimiento RNF-02.

5.4.4. Fallo de un servicio

La llamada continúa aunque un servicio no responda.

Si ILSA no responde, no hay traducción. La cámara de la llamada sigue como cámara de la persona. La extensión no publica un lienzo vacío en su lugar, y el indicador local muestra que el motor no está disponible.

Si el servicio de archivos no responde, vale la regla de la Sección 5.3.4. El esqueleto queda en la postura de espera, la interfaz avisa, y las glosas siguen disponibles como texto.

5.5. Patrones de diseño de software aplicados

El diseño separa las responsabilidades del cliente local, de la extensión y del servicio remoto. Cada patrón acota un problema distinto: la política de admisión de glosas, la interfaz única hacia el motor, la conversión entre contratos y el modo de la sesión. El efecto de cada decisión queda en el componente dueño. La red no filtra rebotes, la extensión no llama a la nube y el servicio remoto no altera el estado de la sesión local. La referencia de estos patrones es (Gamma et al., 1994).

- **Estrategia (*Strategy*), en el filtro de repetición.** El clasificador propone una glosa y una confianza. No decide si esa glosa entra al enunciado. Esa decisión queda en un objeto de política, independiente de la red. La política distingue tres categorías del léxico: los dígitos admiten repetición, las letras admiten hasta dos apariciones consecutivas y el resto del vocabulario rechaza la repetición inmediata. El acumulador de enunciado consulta la política y conserva el reloj de pausa Un cambio de la regla de repetición no altera el modelo ni el contrato remoto.

- **Fachada (*Facade*), en la API local de ILSA.** La extensión no invoca por separado al clasificador, a la compuerta, al acumulador ni al cliente de la nube. El motor expone un único contrato HTTP en el bucle local: sesión, ingesta de la seña, aviso de actividad, cierre de enunciado, estado y diagnóstico. La fachada oculta el orden interno del pipeline y concentra el acceso del navegador.

- **Adaptador (*Adapter*), en dos fronteras.** La extensión adapta los medios de Google Meet al contrato local. La cámara y el lienzo de la llamada se presentan como fotogramas y subtítulos, y el audio o el texto del oyente se presentan como una oración. En la otra frontera, ILSA adapta el enunciado cerrado al contrato del servicio remoto: una cadena de glosas en `/sordo` o en `/translate`, o una oración en `/hearing`. Con las glosas del canal inverso, ILSA arma el nombre de archivo y consulta `/tposes`. Si esa glosa no tiene archivo, las letras salen por `/fingerspell`. La respuesta remota vuelve a entrar por ILSA. Ni el traductor ni el servicio de archivos conocen Meet ni la extensión.

- **Estado (*State*), en el modo de sesión.** La sesión admite dos modos excluyentes: señante y oyente. En modo señante el motor acepta señas y cierra enunciados hacia el canal directo. En modo oyente acepta el texto del interlocutor, consulta el canal

inverso y pide los archivos de seña. La misma fachada cambia de comportamiento según el modo, sin una segunda extensión. El temporizador de pausa no es un estado del clasificador: pertenece al acumulador y solo declara el cierre del enunciado.

5.6. Diseño conceptual de la arquitectura escalable de producción en la nube

Las pruebas de este trabajo usan una instancia de Cloud Run por servicio. Esta sección no describe esa instancia. Describe la proyección para varias videollamadas a la vez, en coincidencia con la restricción RC-02.

El diagrama, en hoja horizontal, está en el Apéndice C (Figura C.1).

La proyección tiene dos servicios detrás de un mismo ingreso. El traductor genera texto. El servicio de archivos entrega un clip de seña. No comparten réplica ni cola.

5.6.1. Ingreso

Las computadoras cliente salen por HTTPS. El ingreso resuelve el nombre, presenta el certificado TLS y filtra tráfico elemental. Detrás opera un balanceador.

- `POST /sign` y `POST /hearing` van al traductor. El criterio es el menor número de conexiones activas, para no cargar una réplica que ya genera un enunciado largo.
- `GET /tposes/{glosa}.pose`, `GET /fingerspell` y la salud de ese servicio van a las réplicas de archivos.

5.6.2. Traductor

El traductor viaja en una imagen de contenedor. Cada réplica monta los binarios GGUF Q4_K_M desde un volumen de solo lectura, así un arranque nuevo no vuelve a bajar los modelos. Un escalador horizontal mira la cola y el tiempo de generación. Si los turnos simultáneos superan a las réplicas activas, el orquestador abre contenedores nuevos. Por la cuantización a 4 bits, cada réplica cabe en menos de 2 GB de RAM y corre en procesador, sin placa de video.

5.6.3. Archivos de seña

Una petición de pose es una lectura, no una generación. El contrato es el de la Sección 5.2.3. Las réplicas no guardan estado y leen la misma carpeta. El escalador de este grupo mira la cantidad de lecturas, no la cola del traductor. Un paso posterior puede pasar la carpeta a un almacén de objetos y dejar una red de contenido delante de las glosas repetidas. La ruta de salud queda aparte de la ruta del archivo. La caché del cliente sigue como primer recurso.

5.6.4. Monitoreo

Un recolector de métricas observa la latencia por oración, la memoria de cada contenedor del traductor, los errores de red y los errores de lectura del servicio de archivos. Esas series ajustan los dos escaladores.

5.6.5. Qué queda fuera de las pruebas

El prototipo no crea el balanceador, ni los escaladores, ni el volumen compartido. ILSA consulta una instancia del traductor, con los dos canales en un solo proceso, y una instancia del servicio de archivos. Esas dos publicaciones se describen en el Capítulo 6. La carga concurrente y el auto-escalado no entran en la validación.

Capítulo 6

Implementación y Desarrollo

6.1. Implementación del subsistema de reconocimiento gestual

El subsistema de clasificación convierte secuencias cinemáticas en glosas de un vocabulario de 100 clases.

6.1.1. Preprocesamiento de datos

Pipeline de captura e ingesta con MediaPipe Holistic. La extracción de datos utiliza MediaPipe Holistic para obtener 33 puntos de pose y 21 puntos por cada mano, conformando un descriptor instantáneo de 225 coordenadas ($33 \times 3 + 21 \times 3 + 21 \times 3$).

Normalización geométrica y corrección de aspecto. Cada fotograma se centra en el punto medio de los hombros y se escala por la distancia inter-hombros. Para corregir la distorsión producida por la relación de aspecto entre cámaras ($4:3$ vs. $16:9$), se implementó un algoritmo de relleno con bandas a 853×480 previa extracción, evitando deformaciones en el espacio cartesiano.

6.1.2. Preprocesamiento y arquitectura neuronal TinySkeletonClassifier

El flujo de entrada se procesa mediante tres etapas: interpolación lineal de fotogramas nulos, recortes de silencio inicial/final y submuestreo uniforme a una longitud fija de $T = 16$ fotogramas.

La red neuronal *TinySkeletonClassifier* está compuesta por:

1. Dos capas de convolución 1D temporal (núcleo 3, 128 canales, BatchNorm, ReLU y Dropout 0,2).
2. Codificación posicional sinusoidal y un Transformer Encoder de 2 capas con 4 cabezas de atención.
3. Capa de agregación por atención (*Attention Pooling*) que asigna un peso ponderado a cada instante según su relevancia informativa.
4. Capa lineal final hacia los logits de clasificación.

6.2. Implementación del subsistema de traducción semántica bidireccional

6.2.1. Selección y especificación de la suite de modelos de lenguaje

Los canales semántico directo (LSA → Español) e inverso (Español → LSA) entrenan la misma suite de modelos de lenguaje pequeños, entre 0,5B y 3B parámetros:

- **Familia Qwen2.5:** variantes de instrucciones de 0,5B, 1,5B y 3B.
- **Familia Llama 3.2 (Dubey et al., 2024):** variante de instrucciones de 1B.
- **Familia SmolLM2 (Allal et al., 2024):** variante de instrucciones de 1,7B.

En el servidor, cada canal arranca con Qwen2.5 de 1,5B y, si ese archivo no está disponible, con Llama 3.2 de 1B. El contrato de esas rutas está en el Capítulo 5.

6.2.2. Régimen común del entrenamiento

Los dos canales comparten el procedimiento. El corpus se parte en 85 % de entrenamiento y 15 % de evaluación, con semilla fija. Cada ejemplo es un diálogo de tres partes: *prompt*, entrada del usuario y respuesta. La carga admite 2048 tokens y guarda los pesos en 4 bits solo en esta etapa.

La adaptación es LoRA (Hu et al., 2022), con rango $r = 16$ y escala $\alpha = 32$, sobre las cuatro proyecciones de la atención y las tres de la red de avance. El modelo de 3B usa lotes de 2 y acumula el gradiente 8 pasos. El resto usa lotes de 4 y acumula 4 pasos. La tasa es 3×10^{-4} para 3B y 5×10^{-4} para los demás. El optimizador es Adam de 8 bits, con decaimiento 0,01, diez pasos de calentamiento y planificador coseno. Usa precisión *bfloat16*.

La pérdida no mira todo el diálogo. Una máscara toma la salida del modelo y la respuesta esperada. El *prompt* y la entrada quedan fuera del gradiente. El error cae solo sobre la respuesta.

Para el entrenamiento, en los dos corpus, las letras sueltas y consecutivas del datatset se funden en un token con mayúscula inicial y los dígitos sueltos en un número. Tres letras iguales seguidas se reducen a dos. Una ráfaga sin separación, como *Mar del Plata*, queda en un solo token *Mardelplata*.

6.2.3. Canal directo (LSA → Español)

El par de este canal es glosas hacia una oración en español rioplatense. La entrada del usuario antepone «Glosas:» a la secuencia ya compacta. La respuesta es la oración esperada. La máscara deja el error sobre esa oración.

La consigna del ajuste pide una sola oración, sin notas ni datos ajenos a la entrada (Massone & Martínez, 2012):

- El español es rioplatense y usa voseo. La oración incorpora artículos, preposiciones y concordancia verbal. La traducción no es palabra por palabra.
- Los pronombres de entrada son `YO`, `VOS`, `EL`, `NOSOTROS` y `ELLOS`. Sin una marca de género al lado, el sujeto queda tácito.

- La letra `A` inmediata al sustantivo o al pronombre marca femenino, y la letra `O` marca masculino. Un parentesco sin esa letra se considera masculino.
- `MIO` y `TUYO` expresan posesivos del español.
- `PASADO` o `AYER` exigen pretérito. `FUTURO` exige tiempo futuro. Sin marca temporal, el verbo queda en presente.
- `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` y `CUANTOS` producen una pregunta directa, con signos de apertura y de cierre.
- `NO` junto al concepto o al verbo produce una negación.
- Un token en capitalización de nombre, o una cifra, se copia tal cual. La consigna prohíbe sustituirlo o alterarlo.
- La entrada sigue el orden *Topic-Comment* (tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y pregunta). La oración de salida reordena esa secuencia según la sintaxis del español.

Al cierre de la corrida, el 15 % reservado se evalúa sobre el binario ya cuantizado. Esa evaluación usa temperatura 0, tope de 24 tokens y penalización de repetición 1,05, y se detiene ante un salto de línea o la marca de fin de turno. Si varias oraciones comparten las mismas glosas, todas cuentan como referencia. La corrida registra acierto exacto, acierto tras normalizar puntuación y mayúsculas, BLEU, ROUGE-L, METEOR y la latencia.

6.2.4. Canal inverso (Español → LSA)

El par invierte al canal directo: la entrada es la oración y la respuesta son las glosas. La entrada antepone «Español:». La máscara deja el error sobre las glosas.

Pide una sola secuencia, separada por espacios, sin notas, sin el prefijo «Glosas:» y sin artículos ni preposiciones del español (Massone & Martínez, 2012):

- Las glosas léxicas van en mayúsculas sostenidas. Un nombre, un apellido o una calle va en capitalización de nombre, como `Patricia` o `Moreno`, y nunca letra por letra. Una cifra, como un documento o una cantidad, va en un solo token numérico.
- El orden es tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y, al cierre, la partícula interrogativa o la negación.
- Los pronombres se explicitan aunque el español los omita. “Tengo un cuchillo.” → `YO CUCHILLO TENER`. El mapeo es yo → `YO`, vos o tú → `VOS`, él → `EL O`, ella → `EL A`, nosotros → `NOSOTROS`, nosotras → `NOSOTROS A`, ellos → `ELLOS O` y ellas → `ELLOS A`.
- El género del parentesco usa el sustantivo en masculino más `O` o `A`: hermana → `HERMANO A`, hija → `HIJO A`, esposa → `ESPOSO A`. Un sustantivo de persona sin marca de género queda en masculino y suma `O` si el español lo explicita.
- `MIO` y `TUYO` cubren los posesivos. “Me llamo X” o “Mi nombre es X” → `YO NOMBRE` más el nombre.

- El pretérito o el copretérito colocan `PASADO` al inicio. El futuro, o la perífrasis "voy a", coloca `FUTURO`. El presente sin marca no agrega token temporal.
- Las preguntas de dónde, quién, cómo, qué, cuándo y cuántos dejan `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` o `CUANTOS` al final. La negación deja `NO` al final, junto al verbo.

El modelo aprende así a emitir el nombre o la cifra ya compactos, no una lista de letras.

La evaluación del 15 % usa temperatura 0, tope de 48 tokens y penalización 1,05. Se detiene ante un salto de línea o la marca de fin de turno. Si la salida trae un bloque de razonamiento, ese bloque se descarta y queda la primera línea. Si varias secuencias de glosas comparten la misma oración, todas cuentan como referencia. Las métricas son las mismas del canal directo.

6.2.5. Cuantización GGUF Q4_K_M y servicio de inferencia

Formulación analítica para la estimación de memoria RAM en tiempo de inferencia. Esta estimación no forma parte del servicio. Acota la memoria del binario Q4_K_M en el procesador, con la ventana de 2048 tokens del entorno publicado:

$$\text{RAM de inferencia} \approx \mathbf{W} + KV_{\text{fp16}}(ctx = 2048) + \text{Buffer de trabajo} \tag{6.1}$$

donde:

- $\mathbf{W}$ es el espacio de los tensores cuantizados, del orden de 0,5 bytes por parámetro en `GGUF Q4_K_M`.
- KV_{fp16} es la caché de claves y valores en `fp16` para 2048 tokens (Pope et al., 2023).
- El buffer de trabajo reserva memoria temporal para las activaciones ($2 \times d_{\text{model}} \times ctx \times 2$ bytes).

6.3. Integración en Google Meet y extensión de navegador

La integración con la plataforma Google Meet se implementó como una extensión de navegador desarrollada bajo la especificación Manifest V3 de Chromium (Google Chrome Developers, 2024). La arquitectura física del código distribuye las responsabilidades en un script en segundo plano (*service worker*), un documento oculto (*offscreen*), una página de entorno aislado (*sandbox*) y dos scripts de contenido (*content scripts*) inyectados sobre el dominio de la videollamada (Google Chrome Developers, 2024). La Figura 6.1 muestra ese reparto. El motor ILSA queda fuera de la extensión.

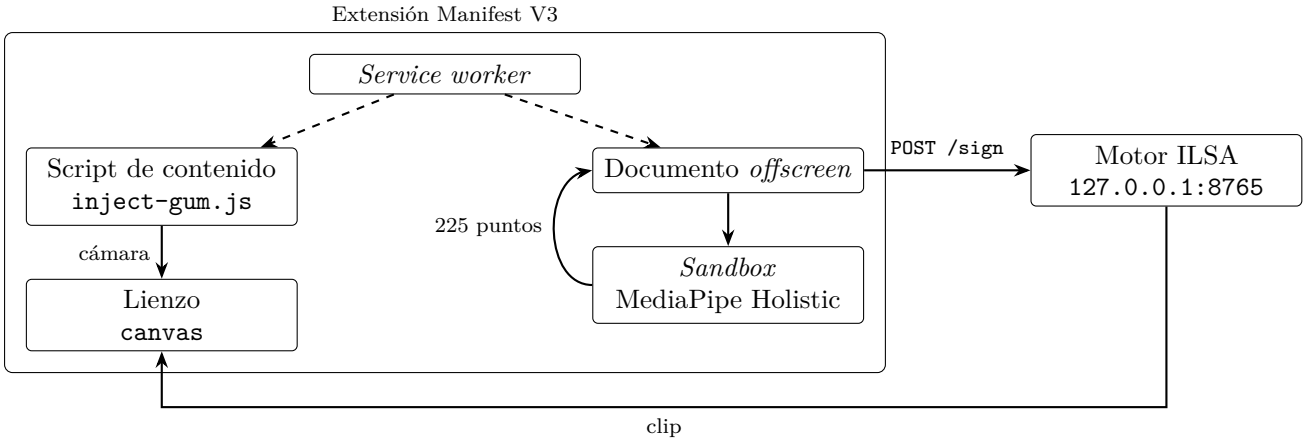


Figura 6.1: Paquetes de la extensión. La extracción corre en el *sandbox*. El lienzo recibe el clip ya armado.

6.3.1. Despliegue en Chrome Manifest V3: aislamiento offscreen y sandbox

El script en segundo plano coordina la mensajería entre el emergente de la extensión, la videollamada de Google Meet y el documento en segundo plano (Google Chrome Developers, 2024). Al detectar la carga activa de una videollamada, el script verifica la disponibilidad del servicio local ILSA en el puerto local (Ramírez, 2018). Al confirmar la conectividad, instancia el documento en segundo plano mediante la API `chrome.offscreen` (Google Chrome Developers, 2024).

Dentro de este esquema, el documento *offscreen* carga un marco iframe configurado como entorno aislado (*sandbox*) con políticas de seguridad relajadas (`Content-Security-Policy`), permitiendo la ejecución de código WebAssembly y la instanciación de los trabajadores auxiliares (*web workers*) de MediaPipe Holistic (Google, 2020; Lugaresi et al., 2019). Para reducir la carga de CPU, las imágenes de la cámara se escalan a 320×240 píxeles antes de transferirse mediante `postMessage` hacia el entorno aislado. La subrutina de visión calcula los 225 puntos articulares tridimensionales y devuelve las coordenadas vectoriales al documento *offscreen*, el cual consolida la ventana temporal de 16 fotogramas y despacha la petición al motor local (Google Chrome Developers, 2024).

6.3.2. Intercepción de la captura de cámara y lienzo virtual

La captura de video se resuelve mediante la inyección del script `inject-gum.js` en el contexto principal de la página web (*MAIN world*), el cual sobrescribe el método estándar `navigator.mediaDevices.getUserMedia` (W3C, 2024).

Cuando la videollamada solicita acceso a la cámara web, el script intercepta la llamada, obtiene el flujo directo del hardware y lo vincula a un elemento `<video>` oculto en el DOM (W3C, 2024). En paralelo, los fotogramas se redibujan sobre un elemento `<canvas>` virtual que opera a 24 cuadros por segundo. A través del método `canvas.captureStream()`, la extensión genera un flujo sintético de medios que se entrega a Google Meet como si fuera el dispositivo físico original (W3C, 2024).

Esta derivación permite superponer la capa visual de subtítulos en la franja inferior del

lienzo sobre un fondo semitransparente antes de emitir la señal a la videollamada (W3C, 2024). De esta manera, el interlocutor oyente puede visualizar las traducciones directamente en la señal de video transmitida por WebRTC sin necesidad de instalar la extensión en su propio navegador (W3C, 2024). La renderización aplica un reflejo horizontal coincidente con la vista previa local y sostiene la visibilidad del subtítulo durante un intervalo de 8 segundos por oración.

La secuencia de eventos que rige la ejecución integrada del canal directo comprende los siguientes pasos:

1. **Intercepción de video y aislamiento:** El script inyectado redirige el flujo de la cámara física hacia el lienzo sintético `<canvas>` mediante `captureStream()` (W3C, 2024) y transmite los fotogramas comprimidos al documento *offscreen* (Google Chrome Developers, 2024).

2. **Extracción cinemática en Sandbox:** El entorno aislado procesa los fotogramas con MediaPipe Holistic y retorna los 225 puntos articulares tridimensionales de torso y manos (Bazarevsky et al., 2020; Google, 2020; Zhang et al., 2020).

3. **Clasificación temporal continua en el motor local (ILSA):** La extensión serializa la secuencia de 16 fotogramas (16×225) y la transmite mediante una petición HTTP POST al endpoint `/sign` del servidor local FastAPI (Ramírez, 2018).

4. **Inferencia local y filtrado heurístico (`RepeatGate`):** El clasificador *TinySkeletonClassifier* genera la predicción. La compuerta de validación evalúa la glosa según su categoría léxica (admitiendo repeticiones en dígitos, un máximo de dos en letras y descartando repeticiones inmediatas en vocabulario general) y actualiza el búfer acumulador (Gamma et al., 1994).

5. **Temporización y cierre de enunciado:** Al transcurrir un intervalo continuo de 4 segundos sin actividad gestual, el acumulador declara el cierre del enunciado y dispara la solicitud de traducción.

6. **Invocación al servicio semántico:** El motor local transmite la lista de glosas mediante `POST /sordo` al servidor remoto. `POST /translate` acepta el mismo cuerpo (Ramírez, 2018).

7. **Emisión concurrente de resultados:** La oración traducida en español se retorna a la extensión para dibujarse como subtítulo sobre el lienzo virtual de Google Meet (W3C, 2024) y, en simultáneo, se envía al hilo secundario de síntesis de voz (`pyttsx3`) para su emisión audible hacia el oyente (Pressman & Maxim, 2014).

6.3.3. Motor local ILSA (FastAPI)

El servidor local se implementó en Python mediante el framework asíncrono FastAPI y se ejecuta sobre un servidor ASGI Uvicorn (Ramírez, 2018). El servicio se enlaza exclusivamente a la interfaz de bucle local (`127.0.0.1:8765`) para denegar conexiones externas no autorizadas.

Para cumplir con las políticas de seguridad del navegador, el middleware del servidor implementa reglas CORS que restringen los orígenes admitidos a `https://meet.google.com` y al identificador de la extensión (`chrome-extension://*`) (Google Chrome Developers, 2024; Ramírez, 2018). Asimismo, responde con la cabecera de acceso a red privada

(`Access-Control-Allow-Private-Network: true`) requerida cuando una página HTTPS realiza peticiones hacia el dominio local (Google Chrome Developers, 2024).

La emisión de voz sintética se desacopló mediante un hilo de ejecución secundario que consume una cola secuencial de mensajes, evitando bloqueos en la atención de las solicitudes HTTP de la API (Pressman & Maxim, 2014). El motor local se empaquetó como un ejecutable independiente de Windows (`ILSA.exe`) utilizando PyInstaller, e incluye una interfaz gráfica desarrollada en Tkinter para permitir la selección de modo (Sordo u Oyente), la configuración de la mano dominante y el control de encendido del servicio.

6.4. Implementación del avatar

6.4.1. Tecnologías empleadas

El servicio de archivos es un proceso aparte del traductor. La imagen parte de Python 3.10 en su variante reducida. Dentro van el programa y la carpeta de poses. El código anterior de captura y clasificación no se copia. El directorio de trabajo del contenedor es `/app`. El proceso arranca con el programa principal y lee el puerto de la variable de entorno. Si esa variable no existe, usa 8080.

La publicación se hizo en Cloud Run, con el nombre `tpose-server`, en la región `southamerica-east1` y en el proyecto `ilsa-511622`. El comando de despliegue parte de la raíz del repositorio, sube el código, construye la imagen y deja el servicio abierto sin token de usuario. El puerto declarado hacia la plataforma es 8080. El proceso, igual, lee el puerto que la plataforma inyecta al arrancar. Los nombres de la carpeta son ASCII. Un archivo con eñe o con tilde en el nombre hace fallar la importación del contenedor, aunque la imagen se construya bien.

El dibujo corre en el lienzo de la extensión de Meet. El motor local, el mismo FastAPI de la Sección anterior, arma la lista de fotogramas. La extensión no estima puntos y no pide archivos por su cuenta. La Figura 6.2 muestra la instancia con la que se probó el prototipo: una sola copia de cada servicio, sin el balanceador de la proyección de producción.

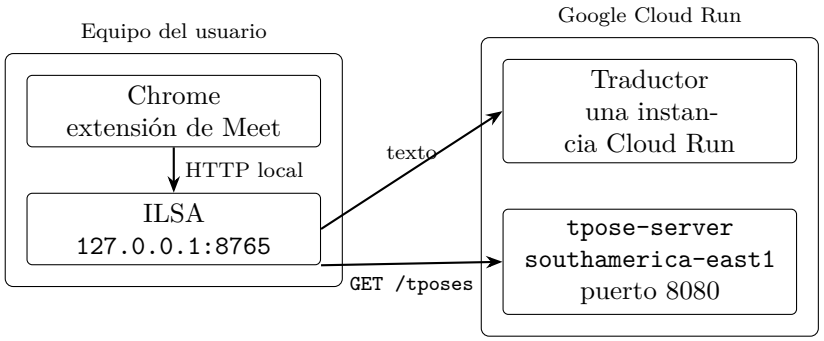


Figura 6.2: Despliegue del prototipo probado. El video no sale del equipo. Cada servicio remoto es una sola instancia.

6.4.2. Vía descartada: retarget hacia un avatar VRM

Antes del diccionario de poses se probó un personaje VRM, con malla y huesos de longitud fija, animado por rotaciones. El fundamento del ensayo es el *retarget*: el paso de posiciones de *landmarks* a rotaciones de otro esqueleto. MediaPipe Holistic entrega

posiciones, y el avatar exige rotaciones sobre su propio esqueleto. El visor se armó con Three.js. Para la mano se probó Kalidokit, una biblioteca de JavaScript para ese *retarget*. Toma los *landmarks* de MediaPipe y calcula rotaciones de Euler del esqueleto y pesos de *blendshape* del rostro. En la mano resume cada dedo en un único ángulo de rizo. También se probó una cinemática inversa que lleva el extremo del hueso hasta el punto medido.

La prueba no se retuvo. El desarrollo no contaba con práctica suficiente de animación tridimensional, y las bibliotecas de *retarget* no sostuvieron de forma estable el paso de la pose al esqueleto VRM. El emparejamiento fallaba con frecuencia porque la pose pierde la profundidad en el eje Z: la coordenada de la mano queda relativa a la muñeca y en una escala distinta del desplazamiento en el plano, de modo que el hueso no recibe una distancia utilizable. Si se respeta la longitud del hueso, la yema no cae sobre el punto medido. Si la cinemática inversa alarga el miembro hasta ese punto, la mano pierde la configuración y, en el alfabeto manual, la letra cambia. Kalidokit, además, confunde configuraciones próximas.

El diccionario de `.pose` evita ese paso: dibuja los puntos medidos y no los convierte en rotaciones de otro cuerpo. Si más adelante se retoma el avatar tridimensional, una línea de mejora es filmar cada seña desde tres ángulos, para reconstruir la profundidad que una sola toma frontal no conserva.

6.4.3. Estructura del pedido

ILSA, en modo oyente, no lee un diccionario de poses del disco de la videollamada. Después de obtener las glosas del canal inverso, normaliza cada nombre y pide `GET /tposes/{glosa}.pose`. `GET /health` solo indica que el proceso está arriba. Un 200 del archivo trae el binario. Un 404 indica que esa glosa no tiene clip.

En el motor local, `POST /hearing` hace las dos consultas: primero la oración hacia el canal inverso y después los archivos. La respuesta trae la oración, las glosas y el clip. En modo señante esa ruta responde 400. La dirección del servicio se reemplaza por configuración, sin recompilar el motor. Los archivos ya bajados quedan en la caché local del equipo. Una glosa repetida no vuelve a viajar por la red.

Si el nombre de una glosa no tiene archivo, ILSA no inventa el movimiento. Pide las letras de esa glosa al mismo servicio, cada una como un archivo. Si el servicio no responde, las glosas siguen disponibles como texto y el clip queda vacío. El tope de la cadena es 24 archivos.

6.4.4. Decisiones técnicas de la reproducción

La Sección 5.3 fija qué se muestra y por qué. Aquí quedan los valores con los que esas reglas se programaron, y cuál de los dos programas hace cada paso. La preparación del clip y el empalme entre glosas corren en ILSA, una vez que los archivos ya llegaron del servicio de poses. La extensión de la videollamada no rehace ese empalme. Recibe la lista de fotogramas ya armada y la dibuja en la mitad derecha del lienzo. El navegador, entonces, no vuelve a calcular el puente: elige, según el tiempo transcurrido, cuál de esos fotogramas le toca mostrar.

Valores del compositor. Antes del recorte, todos los clips se llevan a 30 fotogramas por segundo. Con esa tasa común, una espera de seis fotogramas dura lo mismo en cualquier seña. Si se dejara cada video a la cadencia con la que fue filmado, seis fotogramas de un

clip a 24 por segundo durarían más que seis fotogramas de un clip a 30, y el empalme perdería la medida del tiempo.

El suavizado es un promedio exponencial. Si x_t es la posición recién medida y p_{t-1} es la pose ya aceptada en el fotograma anterior, la pose que se conserva es

$$p_t = \alpha\, x_t + (1 - \alpha)\, p_{t-1}, \tag{6.2}$$

con $\alpha = 0{,}50$. La medición nueva y la pose anterior pesan lo mismo. Ese peso es único para todo el diccionario.

El recuadro común mide 720×780 píxeles. El cuerpo se escala para que el ancho de hombros quede en 250 píxeles. Así, dos tomas de distinto encuadre ocupan el mismo espacio cuando se pegan una detrás de la otra.

El puente dura entre 4 y 8 fotogramas. Si las muñecas del final de una seña y del inicio de la siguiente ya están cerca, se usan 4. Si están lejos, se llega a 8, para que la mano no salte. Un punto cuya confianza queda por debajo de 0,15 sale en la lista como ausente, y el trazo del lienzo no lo une con el punto anterior.

La entrada, desde la postura de espera hasta el primer fotograma de la seña, y la salida, de vuelta a esa espera, duran cerca de seis fotogramas de la tasa de 30. A esa cadencia, cada uno de esos tramos ocupa cerca de un quinto de segundo. La secuencia se corta en el techo fijado en el diseño: 24 glosas y 1200 fotogramas.

Postura de espera. No hay un archivo aparte llamado reposo. El cuerpo y las manos de la espera se copian de los primeros fotogramas de una seña de referencia, el tramo en el que la persona todavía no empezó el gesto, y se escalan al mismo ancho de hombros del lienzo. El antebrazo queda casi horizontal, levemente hacia abajo, y la mano sigue esa línea.

Reflejo de la miniatura local. La videollamada muestra la miniatura de quien usa el sistema como un reflejo. La extensión deshace ese reflejo solo en ese video local. El esqueleto que viaja hacia la otra persona no se invierte. Las dos vistas quedan iguales: la cámara en la mitad izquierda y el esqueleto, sin darlo vuelta, en la mitad derecha.

6.5. Instancia de validación del prototipo mediante túnel seguro

La validación funcional del microservicio semántico remoto se realizó mediante un túnel seguro cifrado (*ngrok*). Esta instancia conectó el motor ILSA local con un servidor semántico desacoplado montado en una máquina independiente, demostrando la viabilidad de la partición híbrida sobre redes residenciales reales sin incurrir en costos de infraestructura elástica comercial.

Capítulo 7

Experimentación, Resultados y Discusión

7.1. Corpus y metodología de adquisición de Datos de LSA

El clasificador no se entrena sobre un corpus público de otra lengua de señas. Un conjunto de prueba en ASL se descartó en las primeras iteraciones: el léxico, la gramática y, en varios ítems, la forma de la seña no coinciden con la LSA, y ese material presentaba además etiquetas inconsistentes y videos espejados. El antecedente local LSA64 aporta un protocolo de captura controlada, con guantes de colores, que no se reproduce en una cámara web doméstica (Ronchetti et al., 2016). El entrenamiento usa, por eso, un corpus propio de LSA.

7.1.1. Definición y validación del vocabulario de 100 clases

El vocabulario objetivo cubre cerca de 100 señas, definidas con la orientación de Miriam Rolls. El recorte léxico apunta a dos situaciones de uso: identificación y verbos de emergencia, por un lado, y asistencia al viajero (lugares, días y familia), por el otro. No se pretende cubrir una conversación abierta.

El entrenamiento no incorpora las 100 señas de manera automática. Una clase entra al modelo solo si alcanza un piso de videos. En julio de 2026 el piso era de 50 videos y el modelo activo tenía 91 clases. En agosto el piso pasó a 60 videos por clase y el conjunto activo llegó a 94, con el alta de `chau`, `tener` y `años`. En septiembre el checkpoint retiene 97 clases: a las 94 se suman `pasado`, `futuro` y `mio`. La letra `ñ` ya formaba parte del conjunto de julio.

Las 97 glosas se agrupan en interrogativos, afirmación y negación, dígitos, alfabeto dactilológico, pronombres, saludos, tiempo, parentesco, lugares y un conjunto cerrado de verbos. El piso de muestras deja fuera del checkpoint a las señas del léxico objetivo que no llegan a 60 videos. Esa exclusión es deliberada: con pocos clips el modelo memoriza al señante de la grabación y la matriz de confusión deja de ser interpretable.

7.1.2. Protocolo de grabación y curación del dataset propio

Los videos de entrenamiento están en 1920×1080 (16:9). La ficha de septiembre contabiliza 6 748 archivos MP4, cerca de 69 clips por seña, y 6 680 tensores `.npy` de forma

$(16, 225)$ en `dataset_landmarks_16frames`. El entrenamiento no usa todos los clips de una clase. Con semilla 42 se mezclan los archivos, se retienen como máximo 60 por clase y el corte es 80 % entrenamiento y 20 % validación, es decir 48 y 12 clips por clase en el régimen de 60 muestras. En el checkpoint del 15/09/2026 eso da 4 656 secuencias de entrenamiento y 1 164 de validación.

El corte es por clip, no por persona. Clips de la misma sesión pueden caer a ambos lados del split. La exactitud de validación queda, por eso, por encima de lo que después mide la cámara: el conjunto de validación no es un señante nuevo ni una óptica nueva.

La aumentación opera solo en entrenamiento y no se guarda en disco. A la escala isotrópica (0,85–1,15) y al ruido gaussiano se suman, desde agosto, rotación 3D (yaw $\pm 15°$, pitch y roll $\pm 8°$), recorte temporal de hasta 15 % por extremo, deformación de velocidad entre 0,80 y 1,25, anulación de hasta 3 fotogramas con interpolación posterior, y ruido distinto en pose (0,012) y en manos. Un multiplicador virtual de 25 repite cada clip con una transformación nueva en cada época. No hay espejado horizontal aleatorio. La validación no recibe aumentación ni repetición virtual.

Dos regrabaciones quedaron incompletas y condicionan los errores finales. La letra `V` se volvió a grabar el 08/09/2026 como pose estática (65 videos de esa fecha). La seña `viernes`, que consiste en la misma configuración de mano con dos golpes al mentón en el eje z, conserva tomas hasta el 17/08/2026 y no se regrabó. El par sigue ambiguo en el dato.

7.2. Evaluación empírica del clasificador gestual

La selección del modelo no sigue la exactitud de validación. El criterio de integración es el acierto top-1 y top-3 frente a la cámara, con una toma por seña, mano diestra y modo `auto`. A lo largo de julio, agosto y septiembre ese criterio descartó la red de mayor capacidad, la búsqueda automática de hiperparámetros y una línea paralela de *transfer learning*, y retuvo el baseline de 16 fotogramas.

7.2.1. Métricas offline vs. rendimiento en cámara real

La Tabla 7.1 resume la brecha que se repite en cada iteración. La validación, sobre clips del mismo corpus y del mismo extractor, supera el 96 %. La cámara, con otra óptica, otra luz y una sola toma, cae entre 15 y 35 puntos según la sesión.

Cuadro 7.1: Exactitud de validación y acierto en cámara del clasificador. Top-1 y top-3 de cámara son crudos: no colapsan homógrafos.

Checkpoint	**Fecha**	**Val. top-1**	**Cámara top-1**	**Cámara top-3**
16 fotogramas, 91 clases	29–30/07	97,36 %	61,5 %	83,5 %
32 fotogramas, 128/4/2	31/07	96,48 %	71,4 %	87,9 %
32 fotogramas, 64/2/1	31/07	97,36 %	67,0 %	90,1 %
Optuna, 32 fotogramas	31/07	97,69 %	66,0 %	84,6 %
Baseline 16f, luz baja, 94 clases	18/08	98,40 %	67,0 %	89,4 %
Baseline 16f, luz alta, 94 clases	28/08	98,40 %	86,2 %	95,7 %
Baseline 16f, 97 clases	15–17/09	98,28 %	77,3 %	94,8 %

Tres factores explican la caída, y no son intercambiables. El primero es el dominio: la

validación reutiliza el extractor y el encuadre de los MP4 de entrenamiento. La cámara de desarrollo no. El segundo es la luz. El mismo baseline de 16 fotogramas pasa de 67,0 % top-1 el 18/08, con luz ambiente, a 86,2 % el 28/08, con mejor iluminación. Ese salto no proviene de un cambio de pesos. El tercero es la varianza de una sola toma. Dos pasadas del mismo conjunto de 36 letras y números, sin cambio de pipeline, movieron el top-1 entre 11 y 19 puntos según el checkpoint. Con una toma por seña, la cámara no distingue efectos menores a unos 14 puntos. Por eso la corrección de relación de aspecto se midió también fuera de línea, sobre los mismos tensores, y no solo con una segunda sesión en vivo.

Una línea paralela de *transfer learning* sobre landmarks alcanzó exactitudes de 0,98 y 0,99 en reportes de 1 084 muestras. Esos reportes incluyen clases de soporte nulo y no cierran el protocolo de cámara que definió el modelo integrado. El checkpoint que se retiene es el TinySkeleton entrenado desde cero.

7.2.2. Impacto de la resolución temporal: 16 vs. 32 fotogramas

En julio, con 91 clases y 50 videos por clase, se compararon 16 y 32 fotogramas bajo el mismo pipeline de recorte y submuestreo. La hipótesis era que más fotogramas conservarían mejor los gestos largos. La Tabla 7.2 y las Figuras 7.1 y 7.2 resumen el resultado.

Cuadro 7.2: Resolución temporal en julio de 2026 (91 clases, 50 videos por clase).

Iteración	**Arquitectura**	**Val. top-1**	**Cámara top-1**	**Cámara top-3**
A, 16 fotogramas	128 / 4 cabezas / 2 capas	97,36 %	61,5 %	83,5 %
B, 32 fotogramas	128 / 4 cabezas / 2 capas	96,48 %	71,4 %	87,9 %
C, 32 fotogramas	64 / 2 cabezas / 1 capa	97,36 %	67,0 %	90,1 %

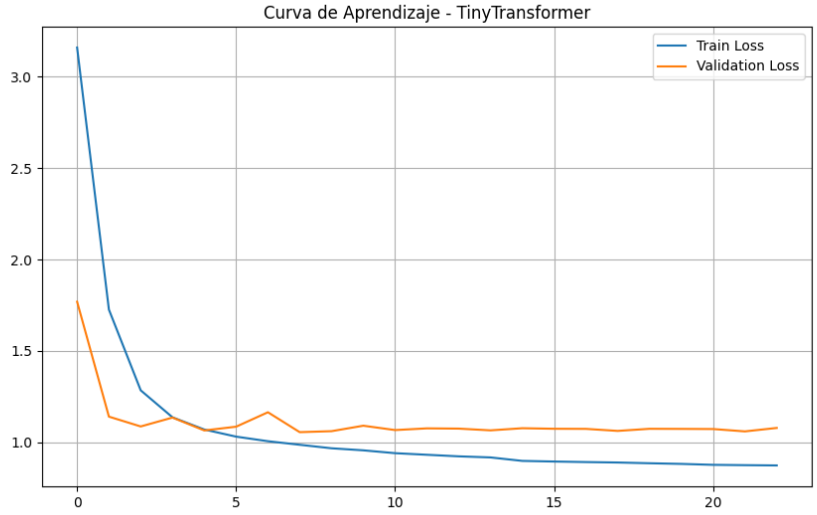


Figura 7.1: Curva de aprendizaje de la iteración A (16 fotogramas, 29/07/2026). La pérdida de validación se estanca cerca de 1,05 desde la época 7, mientras la de entrenamiento sigue en descenso.

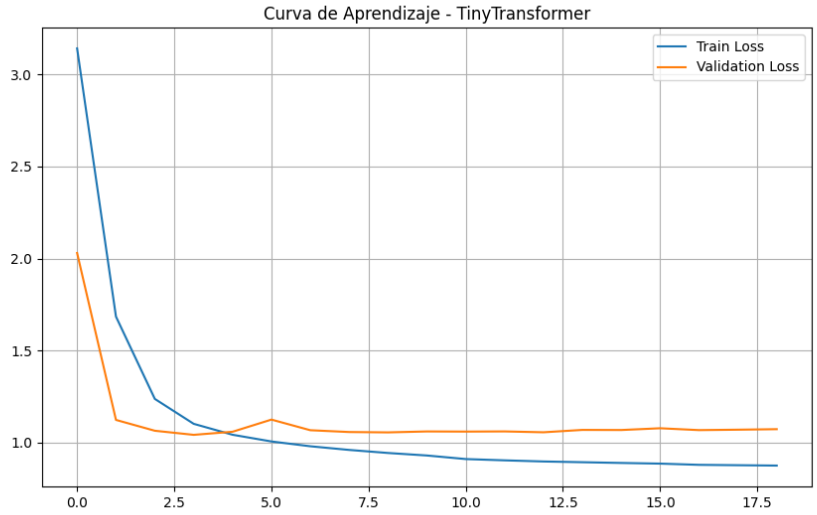


Figura 7.2: Curva de aprendizaje de la iteración B (32 fotogramas, 128 dimensiones, 4 cabezas y 2 capas, 31/07/2026). La pérdida de validación se estabiliza cerca de 1,04 desde la época 4.

En esa semana, 32 fotogramas suman 9,9 puntos de top-1 en cámara respecto de 16 (71,4 % contra 61,5 %). La red más chica de 32 fotogramas no mejora el top-1 y sí el top-3, que llega a 90,1 %. La exactitud de validación, en cambio, casi no se mueve. Más contexto temporal ayuda en vivo con el corpus de julio, y no cierra la brecha con la validación.

A partir de agosto el corpus pasa a 60 videos por clase, la aumentación se amplía y la comparación en cámara, con luz pareja, favorece otra vez a 16 fotogramas. El baseline de 16 fotogramas, 128 dimensiones, 4 cabezas y 2 capas supera en top-1 a las variantes de 12 y de 10 fotogramas. El modelo integrado vuelve a 16 fotogramas. La lectura de julio no se anula: con poco dato, alargar la secuencia mejoró la cámara. Con más dato y con el criterio de producto (top-1 en vivo, letras y dígitos incluidos), la secuencia corta generaliza mejor y evita fotogramas repetidos en las señas estáticas.

7.2.3. Búsqueda automatizada de hiperparámetros con Optuna y Análisis de sobreajuste

Optuna se usó en dos rondas, siempre con validación cruzada de 3 pliegues sobre el conjunto de entrenamiento y con poda de ensayos pobres (Akiba et al., 2019). El objetivo del estudio es la pérdida de validación, no el acierto en cámara.

La ronda de julio, sobre 32 fotogramas y 40 ensayos, elige 128 dimensiones, 8 cabezas, 2 capas, dropout 0,30, lote de 16 y una tasa de aprendizaje de $3{,}77 \times 10^{-5}$. El reentrenamiento con esos valores baja la pérdida de validación a 0,087 y sube la exactitud offline a 97,69 %. En cámara, la corrida oficial da 66,0 % top-1 y 84,6 % top-3, por debajo de la iteración manual B en ambas métricas. Una primera pasada de esa noche se descartó porque una seña se cortó al salir la mano del cuadro.

La ronda de agosto (estudio `tinyskeleton_lsa_v2`, 40 ensayos) libera también la longitud temporal, con candidatos de 8, 12 y 16 fotogramas. El mejor ensayo queda en 12 fotogramas, 256 dimensiones, 2 cabezas, 2 capas y dropout 0,57, con pérdida media de validación 0,137. En cámara, con la misma luz que el baseline del 28/08, ese checkpoint

rinde 80,9 % top-1 crudo y 97,9 % top-3. El baseline de 16 fotogramas, en esa sesión, rinde 86,2 % top-1 y 95,7 % top-3. Optuna gana cerca de 2 puntos de top-3 y pierde cerca de 4 de top-1, sobre todo en letras. Una tercera red, de 10 fotogramas, 256 dimensiones y 3 capas, alcanza 99,02 % de exactitud offline y 77,7 % top-1 en cámara. Una evaluación apareada del 31/08, con la misma toma inferida en las tres redes, confirma el orden: el baseline queda primero en top-1 y también en top-3. La red de 3 capas dispara confianzas cercanas a 1,0 en errores.

El sobreajuste de las curvas de julio es visible en las Figuras 7.1 y 7.2: la pérdida de entrenamiento sigue en descenso y la de validación se estanca por encima de 1. Ese patrón se atenúa en los reentrenamientos de agosto y septiembre, con más dato, más aumentación y parada temprana. La curva final (Figura 7.3) desciende en ambos conjuntos y mantiene una brecha chica. La falla que permanece no es una divergencia de la curva offline. Es el cambio de dominio hacia la cámara.

7.2.4. Diagnóstico de confusiones mediante feature engineering y separabilidad

Con la arquitectura ya elegida, la iteración de septiembre no agrega capacidad a la red. Construye un descriptor de 26 medidas, calculado al vuelo sobre el vector de 225 dimensiones, para responder dos preguntas: qué pares están geométricamente juntos, y si un fallo de cámara también existe en el corpus.

El descriptor tiene tres familias. Seis medidas de geometría mano–rostro, en anchos de hombro, distinguen el lugar de apoyo. Seis medidas de trayectoria de muñeca (desvío y rango en x, y y z) distinguen una letra estática de la seña dinámica que reutiliza esa forma de mano. Catorce medidas de configuración de dedos (extensión, curvatura y separación), normalizadas por el largo de la mano y no por los hombros, distinguen formas que ocupan el mismo lugar y casi no se mueven. Estas 14 se agregaron después de comprobar que las 12 primeras eran ciegas a la forma de la mano.

El descriptor no se concatena al tensor de entrenamiento. El espejado, la aumentación 3D y el recorte temporal asumen que todo el vector son ternas (x, y, z). Intercalar 26 escalares rompería esas operaciones. El cálculo queda fuera del `.npy` y no entra al modelo en esta iteración.

La separabilidad de un par se mide con el AUC de Mann–Whitney sobre cada feature (Mann & Whitney, 1947). El índice informado es $\text{sep} = |\text{AUC} - 0{,}5| \times 2$: vale 1 cuando las dos distribuciones no se solapan y 0 cuando coinciden. En paralelo, un 1-NN con exclusión del propio clip, sobre el tensor $(16, 225)$ aplanado y sobre 6 680 clips, alcanza 88,4 % de acierto. Ese número es optimista, porque clips de la misma sesión se parecen entre sí. Lo útil es qué pares se cruzan.

Los pares duros del alfabeto y de los dígitos se separan con las features de dedos: `2` y `5` por la separación entre medio y anular (sep = 1,00), `D` y `O` por la curvatura del índice (0,97), `M` y `N` por la curvatura del meñique (1,00). El control negativo es `O` y `O`, que permanece en 0,53: comparten forma y movimiento, y ninguna medida del descriptor los separa. `V` y `viernes` quedan en 0,28 de movimiento, coherente con la regrabación incompleta de `viernes`.

El cruce con la cámara del 17/09 marca la mayoría de los fallos como ejecución o captura, no como ausencia de señal en el corpus. `lunes` frente a `L`, `donde` frente a `F` y `años` frente a `G` se separan en los `.npy`. Cuando en vivo la seña dinámica se ejecuta sin el agite, el modelo emite la letra. El descriptor sirvió para decidir qué regrabar y qué no cablear a

la red. No se incorporó al tensor.

7.2.5. Impacto cuantitativo de la corrección de aspect ratio

El efecto del relleno 16:9 se midió primero fuera de línea, sobre 36 tomas ya grabadas en 4:3 y sobre el checkpoint del 29/08. Multiplicar la coordenada y por 4/3 es la corrección analítica de la distorsión. La Tabla 7.3 muestra el resultado, determinista y sobre las mismas tomas.

Cuadro 7.3: Corrección de relación de aspecto sobre 36 letras y números ya grabados (checkpoint del 29/08).

Variante	**Top-1**
Captura original 4:3	66,7 %
Corrección solo en y ($\times$1,333)	88,9 %
Corrección en y y en z	75,0 %

La corrección en y suma 22,2 puntos. Ocho de los diez cambios de predicción van hacia la clase correcta. El caso extremo es `I`: pasaba a `ojo` con confianza 1,000 y pasa a `I` con confianza 1,000. Corregir también z empeora el resultado, en línea con la medición de que el relleno no debe alterar ese eje. De los 12 fallos originales quedan cuatro: `E`, `G`, `O` y `V`. `O` es el homógrafo real con `0`. `V` correspondía a la grabación anterior, ya reemplazada en el reentrenamiento de septiembre.

La variante sin corrección reproduce el 66,7 % de la sesión en vivo de ese modelo. La carga de pesos y el armado del tensor coinciden entre la cámara y la inferencia sobre `.npy`. La divergencia que todavía aparezca en otro cliente queda localizada en la extracción de landmarks, no en la red.

En la evaluación del 17/09, ya con relleno activo (640 $\times$ 480 a 853 $\times$ 480) y con el baseline de 97 clases, `I` sale `I` con confianza 0,89. En las dos pasadas del 15/09, sin relleno, los tres modelos la enviaban a `ojo` con confianza cercana a 1. `T`, `E` y `G` también aciertan en la sesión con relleno. El 88,9 % de las 36 letras y números coincide en valor con una pasada previa en 4:3, pero no es el mismo resultado: se gana `I` y se pierde `V`, porque el vocabulario ahora contrasta esa letra con `viernes`.

7.2.6. Desempeño final del clasificador baseline en vivo

El modelo que se integra es el baseline de 16 fotogramas, 128 dimensiones, 4 cabezas, 2 capas y dropout 0,4, con 97 clases. El checkpoint del 15/09/2026, entrenado 35 épocas con el extractor de Python, alcanza 98,28 % de exactitud top-1 en validación y una pérdida de 0,062. La evaluación en cámara del 17/09, una toma por seña, MediaPipe 0.10.21 y relleno 16:9, da 75 aciertos sobre 97 (77,3 % top-1) y 92 sobre 97 (94,8 % top-3). Cinco señas quedan fuera del top-3: `donde`, `ellos`, `familia`, `nosotros` y `repetir`. Las 36 letras y números llegan a 88,9 % top-1 y 100 % top-3. Si se acepta el homógrafo `O`$\equiv$`0` como acierto, el top-1 de las 97 sube a 78,4 %. Colapsar además las parejas letra–seña dinámica llevaría el índice a 83,5 %, y ese colapso no es el protocolo objetivo: el sistema debe separarlas.

La confianza media es 0,971 en los aciertos y 0,833 en los fallos. El modelo se equivoca con seguridad alta. Un umbral de 0,75 no filtra la mayor parte de esos errores.

Un reentrenamiento del 21/09/2026 conserva la misma arquitectura y el mismo vocabulario, y cambia el extractor al build de Holistic en navegador. Corre 32 épocas, con pérdida

de validación 0,052 y exactitud top-1 de 99,14 %. Las Figuras 7.3 y 7.4 corresponden a esa corrida, que es la última curva y la última matriz archivadas. La sesión de cámara del 17/09 no evalúa estos pesos: evalúa el checkpoint de Python del 15/09 (98,28 %). La matriz de la Figura 7.4 es casi diagonal porque describe la validación offline, no los fallos de cámara.

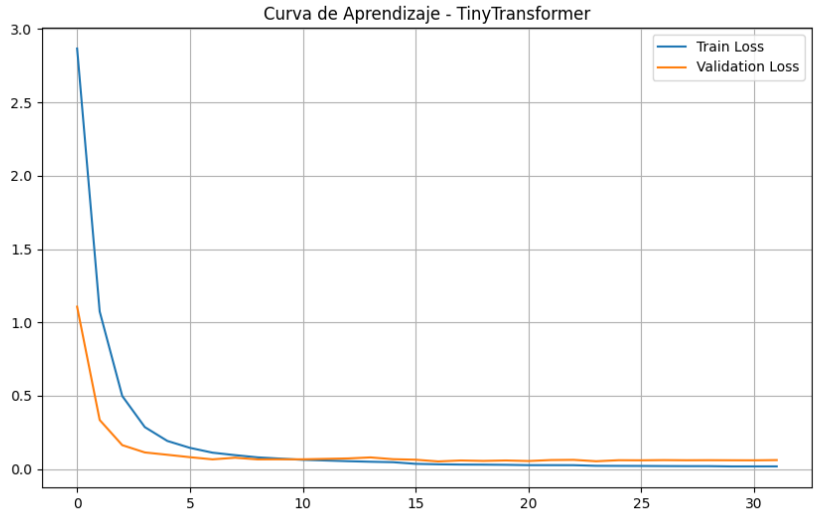


Figura 7.3: Curva de aprendizaje del reentrenamiento del 21/09/2026: 16 fotogramas, 128 dimensiones, 4 cabezas, 2 capas y 97 clases. La pérdida de validación mínima archivada es 0,052 (exactitud top-1 de 99,14 %).

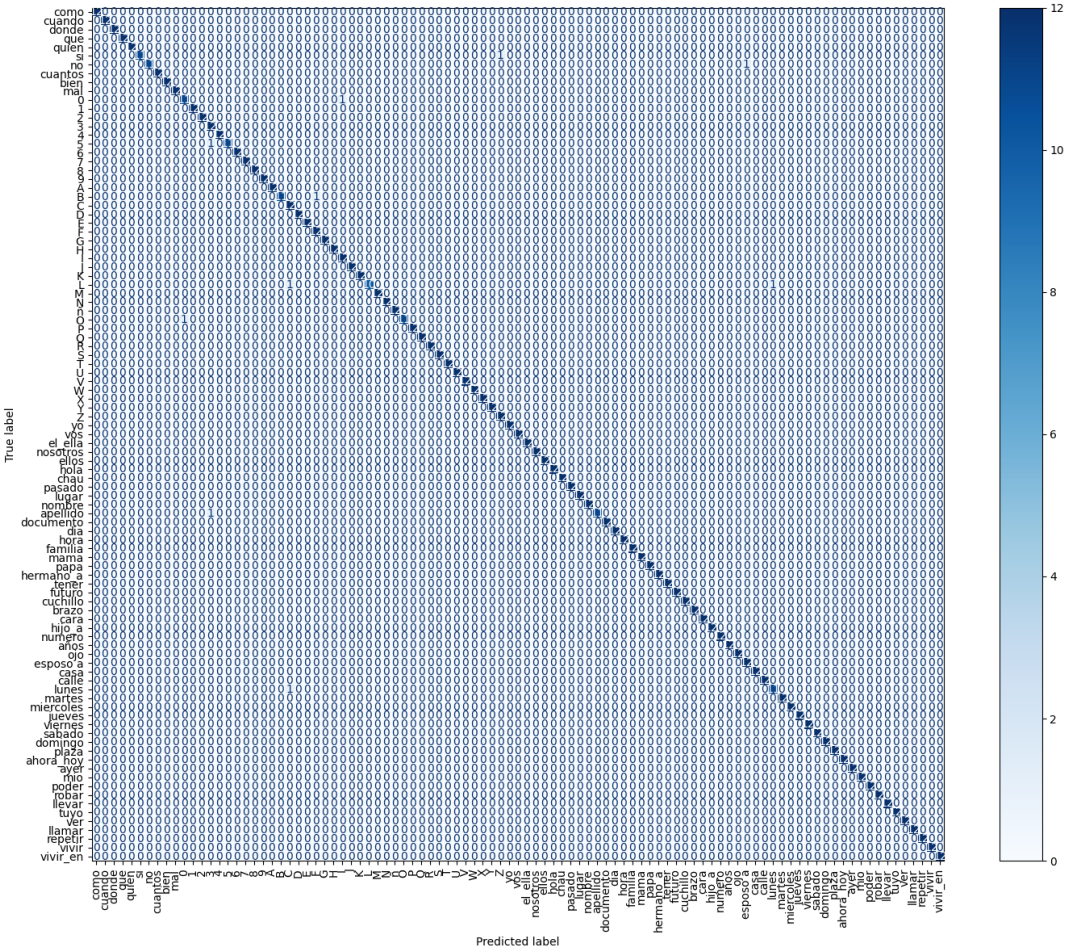


Figura 7.4: Matriz de confusión en validación del mismo reentrenamiento (21/09/2026, 97 clases). La masa se concentra en la diagonal. Los errores de cámara del 17/09 no aparecen en esta figura.

Los bloqueos que la sesión del 17/09 deja abiertos son cuatro. `V` y `viernes` se intercambian en ambos sentidos, porque `viernes` no se regrabó. `O` y `O` siguen como el mismo gesto para el dato disponible (sep = 0,53). Las señas dinámicas que se ejecutan sin agite caen en la letra correspondiente, aunque el corpus sí las separa. `pasado`, clase nueva, atrae a `ayer` y a `cuando`. Diecisiete de los 22 fallos quedan clasificados como captura o ejecución, dos como ambigüedad de `V`/`viernes` y tres en el límite (`O`/`O`, `años`/`G` y `ojo` frente a `vivir`, este último con confianza 0,53 contra 0,47).

Sobre las 94 señas en común con la evaluación del 31/08, el baseline de agosto tenía 86,2 % y esta corrida 77,7 %. El descenso no se atribuye a la red en forma aislada: cambian el vocabulario, la relación de aspecto de la captura y la letra `V`. El cambio atribuible al relleno es la recuperación de `I` y de `T`. El siguiente paso de dato, no de arquitectura, es regrabar `viernes` y repetir las tomas frágiles. Con una sola toma, el porcentaje global sigue sujeto a la varianza ya medida.

7.3. Evaluación de los módulos de traducción semántica

7.3.1. Benchmark comparativo de modelos SLM (Familias Qwen2.5, Llama 3.2 y SmolLM2)

7.3.2. Evaluación cuantitativa: BLEU, ROUGE-L, METEOR y exactitud

7.3.3. Análisis de simetría en el aprendizaje: traducción directa vs. estructuración inversa

7.3.4. Compromiso entre calidad lingüística y latencia de inferencia

7.4. Evaluación del sistema integrado y rendimiento en tiempo real

7.4.1. Validación funcional de extremo a extremo en videollamadas reales

7.4.2. Análisis de latencias totales por etapa del pipeline

7.4.3. Consumo de recursos de hardware: máquina de desarrollo vs. máquina objetivo

7.4.4. Discusión técnica y lecciones de ingeniería aprendidas

Capítulo 8

Conclusiones y trabajo futuro

8.1. Conclusiones

8.1.1. Cumplimiento de objetivos del proyecto

8.1.2. Aportes técnicos y de accesibilidad

8.1.3. Impacto social en el marco de la Ley 27.710

8.2. Trabajo futuro

8.2.1. Implementación física del clúster semántico escalable

8.2.2. Expansión del corpus multiseñante y mitigación de variabilidad

8.2.3. Incorporación de rasgos no manuales (expresiones faciales)

8.2.4. Generalización a nuevas plataformas de comunicación virtual

Bibliografía

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework, 2623-2631.

Alibaba Cloud. (2024). Qwen2.5: A Family of Large Language Models.

Allal, L. B., Lozhkov, A., Bakouch, E., von Werra, L., & Wolf, T. (2024). SmolLM2: When Small Goes Big.

Barán, V. (2018). LSApp: Uniendo dos mundos.

Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-device Real-time Body Pose Tracking.

Blessit. (2024). Interpretación Lengua de Señas — Central de intérpretes.

Confederación Argentina de Sordos. (2024). Señario de términos y expresiones en Lengua de Señas Argentina.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Gerganov, G. (2023). *llama.cpp: Port of Facebook's LLaMA model in C/C++*. https://github.com/ggerganov/llama.cpp

Google. (2020). MediaPipe: A Framework for Building Perception Pipelines.

Google Chrome Developers. (2024). Overview of Manifest V3.

Hand Talk. (2025). Acessibilidade digital em línguas de sinais.

HETAH. (2024). Avatar traductor a lengua de señas.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *Proceedings of the International Conference on Learning Representations*.

INJU. (2024). Diccionario de Señas.

Li, D., Rodriguez, C., Yu, X., & Li, H. (2020). Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison. *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, 1459-1469.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. *Third Workshop on Computer Vision for AR/VR at IEEE CVPR*.

Mann, H. B., & Whitney, D. R. (1947). On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, *18*(1), 50-60.

Massone, M. I. (1993). El número y el género en la Lengua de Señas Argentina. *Signo & Seña*, *2*, 73-99.

Massone, M. I., & Martínez, R. A. (2012). *Curso de Lengua de Señas Argentina: Parte 1 y 2*. Cultura Sorda. http://www.cultura-sorda.eu

Mindlin, I. (2021). *Reconocimiento de Lengua de Señas con redes neuronales recurrentes* [Tesis de grado]. Universidad Nacional de La Plata, Facultad de Informática. http://sedici.unlp.edu.ar/handle/10915/129853

Ministerio de Justicia de la Nación. (2024). Características de la comunicación por videollamadas — Línea 144.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Zhang, C., Nado, Z., Jaitly, N., Vaswani, A., et al. (2023). Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, *5*, 606-624.

Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th). McGraw-Hill.

Ramírez, S. (2018). *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python*. https://github.com/fastapi/fastapi

República Argentina. (2023). Ley 27.710 — Lengua de Señas Argentina.

Ronchetti, F., Quiroga, F., Estrebou, C., Lanzarini, L., & Rosete, A. (2016). LSA64: An Argentinian Sign Language Dataset. *XXII Congreso Argentino de Ciencias de la Computación (CACIC)*, 794-803.

SignAll Technologies. (2026). About SignAll Technologies.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *arXiv preprint arXiv:1706.03762*.

W3C. (2024). Media Capture from DOM Elements: `HTMLMediaElement.captureStream()`.

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). MediaPipe Hands: On-device Real-time Hand Tracking.

Apéndice A

Vocabulario del clasificador

El clasificador distingue 100 clases. Cada clase cuenta con 100 clips. Miriam Rolls delimitó el conjunto. La etiqueta es el nombre que devuelve el modelo. La columna de lectura solo expande esa etiqueta: no es una traducción al español ni una definición de la seña.

Las 27 clases del alfabeto manual son las letras A–Z y Ñ. Ñ es una clase propia, distinta de N. Las 10 clases de dígitos son 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9. Las 63 clases restantes quedan en los cuadros de abajo.

Cuadro A.1: Interrogación, afirmación y negación.

Etiqueta	**Lectura**
`como`	cómo
`cuando`	cuándo
`donde`	dónde
`que`	qué
`quien`	quién
`si`	sí
`no`	no
`cuantos`	cuántos
`bien`	bien
`mal`	mal

Cuadro A.2: Pronombres y saludo.

Etiqueta	**Lectura**
`yo`	yo
`vos`	vos
`el_ella`	él o ella
`nosotros`	nosotros
`ellos`	ellos
`hola`	hola
`chau`	chau

Cuadro A.3: Tiempo.

Etiqueta	**Lectura**
`pasado`	pasado
`futuro`	futuro
`dia`	día
`hora`	hora
`ahora_hoy`	ahora u hoy
`ayer`	ayer
`lunes`	lunes
`martes`	martes
`miercoles`	miércoles
`jueves`	jueves
`viernes`	viernes
`sabado`	sábado
`domingo`	domingo

Cuadro A.4: Lugar, identificación y parentesco.

Etiqueta	**Lectura**
`lugar`	lugar
`casa`	casa
`calle`	calle
`plaza`	plaza
`nombre`	nombre
`apellido`	apellido
`documento`	documento
`numero`	número
`años`	años
`familia`	familia
`mama`	mamá
`papa`	papá
`hermano_a`	hermano o hermana
`hijo_a`	hijo o hija
`esposo a`	esposo o esposa

Cuadro A.5: Cuerpo, objeto, posesivos, verbos y auxilio.

Etiqueta	**Lectura**
`brazo`	brazo
`cara`	cara
`ojo`	ojo
`cuchillo`	cuchillo
`mio`	mío
`tuyo`	tuyo
`tener`	tener
`poder`	poder
`vivir`	vivir
`vivir_en`	vivir en
`ver`	ver
`llamar`	llamar
`llevar`	llevar
`ayudar`	ayudar
`repetir`	repetir
`robar`	robar
`policia`	policía
`gracias`	gracias

Apéndice C

Arquitectura escalable de producción en la nube

La Figura C.1 pertenece a la Sección 5.6. La hoja va en horizontal para que el ingreso, el traductor y el servicio de archivos queden en una sola vista. El diagrama describe la proyección para varias videollamadas a la vez.

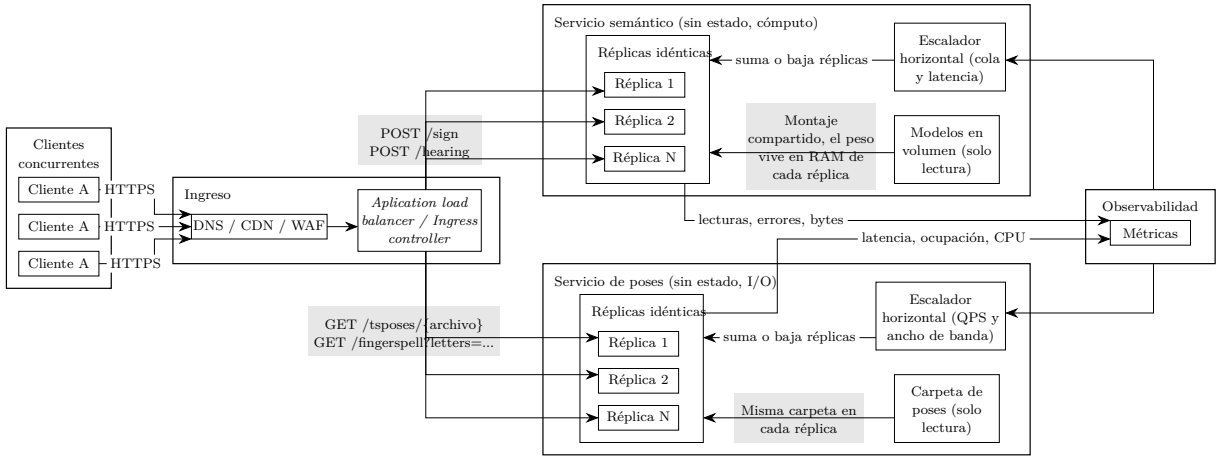


Figura C.1: Diagrama de la arquitectura del servidor, balanceo de carga y escalabilidad de servicios sin estado.